

DISTRIBUTED LEDGER GOVERNANCE, TOKENIZATION, AND MULTI-ASSET REGISTRY SYSTEMS WITH MATHEMATICALLY ENFORCED BACKING

FIELD OF THE INVENTION

The present invention relates generally to distributed ledger technology, smart contract security, decentralized governance, asset tokenization, and automated compliance systems. More specifically, it relates to: (a) multi-signature approval systems embedded directly within smart contract code; (b) systems and methods for tokenizing physical precious metal assets stored in vaults with mathematically enforced backing ratios; and (c) systems and methods for tokenizing diverse physical real-world assets using configurable asset class registries, conditional claim instruments, and shared compliance, governance, and fee components.

BACKGROUND OF THE INVENTION

This application relates to distributed ledger technology for governance, tokenization of real-world assets, and multi-asset registry management. Interest has grown in using distributed ledger platforms to manage physical assets, enforce compliance, and coordinate multi-party approvals for institutional use cases.

Governance of smart contract actions presents technical challenges. Multi-party approval mechanisms must balance security with operational efficiency, avoid deadlock conditions where neither approval nor disapproval can proceed, and enforce deterministic execution ordering to prevent race conditions. Mechanisms are needed for handling stalled or abandoned requests without blocking subsequent operations.

Tokenization of physical assets such as precious metals stored in custodial vaults raises further challenges. Maintaining a provable relationship between circulating digital tokens and underlying physical assets requires atomic state changes that update token supply, collateralization status, and cryptographic proofs of reserves in a single transaction. Storage fee structures, compliance-gated access, and legal forfeiture processes must operate consistently with these backing guarantees.

At a broader level, extending tokenization beyond a single asset class to support diverse real-world assets -- each with different units of measure, quality metrics, and valuation formulas -- requires a configurable registry architecture. Such an architecture should permit new asset classes to be registered and tokenized using shared infrastructure for governance, compliance verification, and proof-of-reserves, without requiring code changes for each new asset type.

The systems and methods described herein address these challenges through three integrated technology layers.

SUMMARY OF THE INVENTION

The present invention provides three interrelated systems that, when combined, deliver a comprehensive platform for governance-backed, compliance-gated, mathematically proven real-world asset tokenization on distributed ledgers.

A first aspect of the invention provides a Guardian Multi-Signature Approval System -- an inheritable, reusable governance framework for distributed ledger smart contracts. The Guardian system provides: an inheritable template pattern enabling any contract to gain multi-signature capabilities through inheritance or composition; symmetric approval and disapproval voting with intelligent threshold calculation using the formula $\text{disapprovalThreshold} = \min(\text{approvalThreshold}, \text{guardianCount} - \text{approvalThreshold} + 1)$ that mathematically guarantees deadlock prevention; sequential execution enforcement requiring $\text{requestId} == \text{nextExecutableRequestId}$ for execution, with narrowly-scoped exceptions for emergency security actions and disapproval cancellation; dynamic threshold management with automatic threshold adjustment when guardian sets change; progressive expiration cleanup with configurable timeout defaulting to seven days; and action-specific configuration allowing different thresholds and guardian sets per action type.

A second aspect of the invention provides a Precious Metal Tokenization System (exemplified by Gold Bars v1.3) -- a distributed ledger-based framework for tokenizing physical precious metals stored in vaults. The system provides: mathematically enforced 1:1 backing where total fungible token supply equals the exact sum of fine metal ounces in all collateralized bar contracts at all times; fine metal ounces calculation supporting multiple purities using $\text{fineMetalOzt} = \text{grossWeightOzt} \times \text{purity} / 10,000$; tiered percentage-based storage fees with asset-type-specific tier structures based on total gross weight holdings per user per asset type per vault; automatic tier recalculation when user holdings cross tier boundaries; risk-based KYC expiration with validity periods of one to three years based on risk level; an information request workflow providing structured off-chain communication with on-chain state transitions; atomic legal forfeiture execution with encrypted court order storage and Guardian verification; deposit and consignment separation with distinct flows for physical movement and on-chain custody placement; Merkle proof-of-reserves with per-leaf structure using SHA-256 hashing over owner, barId, vaultId, fineGoldOzt, and a cryptographically random 256-bit salt for privacy; and collateralization with automatic public claim enablement.

A third aspect of the invention provides a Conditional Claim Protocol (CCP) -- a distributed ledger-based framework for tokenizing diverse real-world assets using shared protocol components. The CCP provides: a configurable asset class registry supporting dynamic registration of new asset classes with per-class metadata schemas, unit calculation formulas, base quantity units, quality factor formats, and divisors; a linear unit-value calculation formula ($\text{unitValue} = \text{baseQuantity} \times \text{qualityFactor} / \text{divisor}$) chosen to reduce attack surface compared to scriptable or oracle-driven valuations; shared protocol components including proof-of-reserves, reusable KYC/compliance proofs, prepaid tiered storage fee escrow, and Guardian multi-party approval; conditional claim instruments where each asset contract represents a right to obtain ownership and control of the physical asset upon satisfaction of specified conditions rather than a custodial IOU; deposit and consignment separation with information-request states including a default seven-day timeout with vault or user cancellation; and optional fractionalization support enabling future splitting of asset contracts into sub-claims.

These three aspects are deeply integrated. The Guardian governance library provides the governance control plane for both the Gold Bars and CCP systems, governing state-changing and administrative actions while permitting designated owner-only actions to execute atomically without

queue-based latency. The Gold Bars system is a specialized, production-ready implementation of the broader CCP framework, optimized for precious metals. The CCP extends this foundation to arbitrary asset classes while sharing the same compliance, fee, and governance infrastructure.

DETAILED DESCRIPTION OF THE INVENTION

I. INTRODUCTION AND SYSTEM OVERVIEW

The following description sets forth specific details in order to provide a thorough understanding of the invention. It will be apparent to those of ordinary skill in the art that the invention may be practiced without some of these specific details. In some instances, well-known structures and techniques have not been described in detail so as not to obscure the invention. The disclosed embodiments are illustrative and not restrictive.

As used herein, the term 'Guardian' refers to a party authorized to approve certain protected actions within a distributed ledger smart contract. The term 'protected action' refers to an action designated as requiring Guardian approval. The term 'owner-only action' refers to an action that may be executed by an asset or contract owner without Guardian approval, typically atomically. The term 'asset contract' refers to a smart contract or ledger object representing a specific physical asset and storing its lifecycle state. The term 'collateralized' refers to a state in which fungible units are minted against an asset and the asset is included in a proof-of-reserves commitment. The term 'conditional claim' refers to a legal rights instrument conferring enforceable rights to obtain full ownership and control of a physical asset once specified conditions are met.

The systems and methods described herein are designed for privacy-preserving distributed ledger networks such as Canton Network using Daml, but are applicable to any distributed ledger technology including Ethereum Virtual Machine (EVM) compatible blockchains. Inventor: Paul West. Assignee: Global Gold Finance, LLC.

FROM GOVERNANCE TO TOKENIZATION

The Guardian governance system described in Section II below provides the foundational trust layer upon which the tokenization systems of Sections III and IV are built. In the Gold Bars system (Section III), the Guardian library governs vault controller choices such as consignment approvals, denial decisions, legal forfeiture execution, emergency vault controls, and proof-of-reserves updates, while owner-only actions such as collateralize, uncollateralize, claim, and transfer execute atomically without Guardian approval to avoid queue-based latency. This same governance architecture extends to the Conditional Claim Protocol (Section IV), where Guardian approval governs asset class registration, custodian onboarding, and administrative actions across all supported asset classes. The shared compliance components -- including reusable KYC/compliance proofs, prepaid tiered storage fee escrow, and Merkle-based proof-of-reserves -- are common to both systems.

II. GUARDIAN MULTI-SIGNATURE APPROVAL SYSTEM

A. Core Architecture and Definitions

A reusable Guardian library provides a governance control plane for smart contracts by designating certain actions as Guardian-governed (requiring multi-party approval before execution) while permitting designated owner-only actions to execute atomically without Guardian approval. The Guardian library is designed to be inherited, imported, or composed into any target smart contract, enabling each contract to have its own independent guardian system.

The Guardian system uses the following core data structures. A `PendingRequest` or `PendingAction` template stores a unique sequential `requestId`, a reference to the parent contract, an `actionType` identifying the type of action being requested, a `submitter` identifying the party that submitted the request, a `submittedAt` timestamp, approval and disapproval sets tracking which guardians have voted, a `finalized` boolean, and action-specific parameters. A `GuardianConfiguration` stores the list of guardian parties, a `defaultThreshold` for all actions, a map of action-specific thresholds, a `requestTimeout` duration after which requests expire (defaulting to seven days), a `nextRequestId` counter, and a `nextExecutableRequestId` pointer to the next request that can execute.

B. Voting Logic with Deadlock-Resistant Disapproval Thresholding

A governance request supports both approval and disapproval votes. Upon a guardian submitting a vote, the system validates the guardian, checks vote type, prevents duplicate voting, updates the relevant count, and evaluates thresholds.

The disapproval threshold is computed as: $\text{disapprovalThreshold} = \min(\text{approvalThreshold}, \text{guardianCount} - \text{approvalThreshold} + 1)$. This formula guarantees the inequality $\text{approvalThreshold} + \text{disapprovalThreshold} > \text{guardianCount}$, making simultaneous approval and disapproval threshold achievement mathematically impossible. The proof proceeds in two cases: if $A \leq G - A + 1$, then $D = A$ and achieving both thresholds requires $2A$ votes from G guardians, but since $2A \leq G + 1$ and each guardian votes at most once, both thresholds cannot be met simultaneously; if $A > G - A + 1$, then $D = G - A + 1$ and achieving both thresholds requires $A + (G - A + 1) = G + 1$ votes from only G guardians, which is impossible.

C. Sequential Execution Enforcement with Optional Out-of-Order Exceptions

Even if an approval threshold is met, execution is permitted only when the request is the next executable request in a sequential queue (`requestId == nextExecutableRequestId`). This enforces deterministic execution order and prevents transaction ordering attacks and race conditions.

The system computes an execution predicate: `CanExecute = (requestId == nextExecutableRequestId) OR canExecuteOutOfOrder(actionType)`. Out-of-order execution is permitted only for: (1) emergency action types (`VAULT_LOCK`, `VAULT_UNLOCK`, `VAULT_RECOVER`, `VAULT_EXECUTE_RECOVERY`) when the approval threshold is met; and (2) disapproval cancellation when the disapproval threshold is met, which prevents malicious or erroneous requests from blocking the queue.

D. Request Expiration, Timeout, and Queue Unblocking

Requests include a configurable expiration time defaulting to seven days. There are two resolution paths for expired or blocking requests: (1) letting the request expire, after which it is marked as expired, cleaned up, and the queue incremented to allow the next request to process; or (2)

submitting a cancel request for the blocking request, which can be executed out of order (pending Guardian approval) for immediate removal. Cancel requests enable prompt unblocking of the sequential queue without waiting for timeout expiration.

E. Threshold Dynamics on Guardian Addition or Removal

When a guardian is added or removed, the default threshold and all action-specific thresholds are checked to confirm that they do not violate the rule requiring threshold to be a maximum of one less than the number of guardians. If any threshold is found to be in violation of this rule, the threshold is decremented to adhere to the rule. Threshold recalculation occurs atomically in the same transaction as the guardian set change. In the case of a guardian removal, the change only affects new requests; all pending requests use the threshold as defined when the request was submitted.

FROM PRECIOUS METALS TO MULTI-ASSET

While the Gold Bars system described above is optimized for precious metals, many of its innovations -- atomic backing, tiered fee escrow, compliance-gated workflows, and Guardian governance -- are general-purpose. The Conditional Claim Protocol described below abstracts these patterns into a configurable framework supporting arbitrary asset classes (real estate, art, commodities, and others) through a dynamic asset class registry, while preserving the same mathematical guarantees and shared infrastructure.

III. PRECIOUS METAL TOKENIZATION SYSTEM (GOLD BARS v1.3)

A. Core Architecture

The Gold Bars system comprises the following core data structures and entities. A GoldBar contract or asset template represents a single physical gold bar and stores: barId (unique identifier), vault (location/custodian identifier), owner (current custodian owner), serialNumber (bar serial number), grossWeightOzt (weight in troy ounces before purity adjustment), purity (expressed in basis points, e.g., 10000 for pure gold), assetType (gold, platinum, silver, etc.), collateralized (boolean indicating whether USG tokens are minted against this bar), publicClaimEnabled (boolean allowing public submission of claim requests), ipfsCID (content identifier for off-chain documentation), legalHash (cryptographic hash of legal documents related to forfeiture), and merkleLeafIndex (position in proof-of-reserves Merkle tree).

A StorageFeeEscrow contract maintains: a vaultId identifying the vault, an assetType, a userId identifying the user, a prepaidAmount (total fees prepaid), a currentTierIndex (user's current fee tier), tierThresholds (gross weight thresholds for tier transitions), tierRates (percentage-based fee rates per tier), lastRecalculationTime (timestamp of last tier recalculation), and userFeeHistory (log of fee calculations for audit purposes).

A KYCProof contract stores: userId, riskLevel (low, medium, high), expirationTime (computed as issuanceTime + riskLevelDuration), issuedAt (timestamp), issuer (KYC provider), and graceEndTime (30 days after expiration, after which restrictions apply). The KYCProof does not store personally identifiable information; metadata only is recorded on-chain.

B. Fine Metal Ounces Calculation

The fine metal ounces (or 'fine gold ounces' for gold specifically) for a given Gold Bar contract is computed using the formula: $\text{fineMetalOzt} = \text{grossWeightOzt} \times \text{purity} / 10,000$. This formula supports multiple purities and asset types. For example, a gold bar with $\text{grossWeightOzt} = 1000$ and $\text{purity} = 9950$ (99.50% pure) yields $\text{fineMetalOzt} = 1000 \times 9950 / 10,000 = 995$ fine ounces. A platinum bar with $\text{grossWeightOzt} = 500$ and $\text{purity} = 10000$ (pure platinum) yields $\text{fineMetalOzt} = 500 \times 10000 / 10,000 = 500$ fine ounces. Purity is expressed in basis points to avoid floating-point arithmetic and ensure deterministic computation across all ledger implementations.

C. Mathematically Enforced 1:1 Backing Invariant

A core invariant of the Gold Bars system is: $\text{totalUSGSupply} == \text{sum}(\text{fineMetalOzt} \text{ in all collateralized GoldBar contracts})$. This invariant is enforced mathematically through atomic mint and burn operations. When a GoldBar is collateralized, its fine metal ounces are summed and an equivalent quantity of USG fungible tokens is minted. Conversely, when a GoldBar is uncollateralized or claimed, the USG tokens are burned and the bar is removed from the collateralized set. All state transitions that affect this invariant (collateralize, uncollateralize, claim, legal forfeiture) occur atomically within a single ledger transaction, ensuring that the invariant is never violated even if transactions are executed out of order or concurrently.

The invariant is further verified through integration with the Merkle proof-of-reserves system, where every collateralized bar contributes exactly one leaf to the Merkle tree. The sum of all fine metal ounces in the tree equals total USG supply. This redundant verification provides cryptographic assurance that the backing invariant is preserved.

D. Merkle Proof-of-Reserves with Privacy-Preserving Leaves

Each Merkle leaf represents a collateralized Gold Bar backing USG tokens. The leaf structure enables users to verify their USG holdings are backed by specific physical gold bars. The canonical leaf structure is: $\text{leaf} = \text{SHA-256}(\text{owner} \parallel \text{barId} \parallel \text{vaultId} \parallel \text{fineGoldOzt} \parallel \text{salt})$, where salt is a cryptographically random 256-bit nonce. Verification includes user verification via $\text{verifyAllocation}(\text{user}, \text{amount}, \text{proof}[])$ proving USG is backed by specific bars, and total supply verification confirming that the sum of all fineGoldOzt in leaves equals total USG supply. The salt prevents third parties from probing whether specific user/bar combinations exist in the tree.

The Merkle tree is updated atomically whenever a bar is collateralized or uncollateralized. Each update triggers a recalculation of affected parent nodes up to the root. The root is signed by the vault to certify the proof-of-reserves state. Users may independently verify the Merkle proof and validate their allocation without trusting any intermediate party.

E. Oracle-Free Physical-to-Digital Bridge

The system does not use Chainlink or any external oracle service. Unlike traditional oracle-dependent systems that fetch external price feeds or attestations, the Gold Bars system uses a vault-submitted blockchain transaction as the sole source of truth for physical-to-digital state synchronization. The vault submits a transaction listing the current inventory of bars, their weights, purities, and serial numbers. This transaction is validated and processed by the smart contract, which updates the on-chain bar registry and triggers proof-of-reserves updates. Multi-layer verification ensures integrity: (1) the transaction is cryptographically signed by the vault, (2) Guardian

multi-signature approval verifies consensus among governance parties before proof-of-reserves updates are committed, and (3) users may independently verify their bars against the Merkle proof.

This design eliminates oracle dependency and the attack surface associated with oracle manipulation. It also avoids the latency and cost of external oracle calls, enabling real-time state synchronization between physical vaults and the distributed ledger.

F. Collateralization with Automatic Public Claim Enablement

When a GoldBar contract is collateralized (USG tokens are minted against it), the `publicClaimEnabled` flag is automatically set to true. This enables any user who possesses the corresponding USG tokens to submit a claim request for the bar, initiated by burning their USG. The owner cannot disable `publicClaimEnabled` while the bar is collateralized, ensuring that USG holders retain enforceable claim rights.

To regain full control and disable public claims, the owner must first uncollateralize the bar, which burns any outstanding USG and removes the bar from the proof-of-reserves tree. Only after uncollateralization is the `publicClaimEnabled` flag released for owner control.

G. Claim, Uncollateralization, and USG Burn Timing

USG is burned when a Gold Bar NFT is claimed. The user requests to claim a Gold Bar NFT; their USG for the claim is locked. The vault reviews the request. Upon approval by the vault, which includes the vault performing KYC and title transfer, the USG is burned and the user receives the Gold Bar NFT. Redemption of physical gold (Gold Bar NFT) requires the user to submit a request for redemption which locks the Gold Bar NFT. The vault reviews the request and communicates with the holder regarding costs. Once the physical gold bar is shipped, the Gold Bar NFT is burned. Issues pertaining to delivery, loss, and damage after shipping are handled using existing real-world procedures. If there are issues prior to shipping, the request can be reverted by either the vault or the user, which closes the request and unlocks the Gold Bar NFT for the user.

H. Tiered Storage Fees with Automatic Tier Recalculation

Storage fees in the Gold Bars system are percentage-based and calculated on the gross weight of assets held by a user in a specific vault for a specific asset type. The fee structure supports multiple tiers with threshold-based transitions. For example, a vault may define: 0-1000 oz at 2% per annum, 1001-5000 oz at 1.5% per annum, 5001+ oz at 1% per annum. When a user collateralizes a bar, the system calculates their new total gross weight and automatically reassesses their tier if they cross a threshold.

Fees are prepaid for a full period (e.g., one year) and stored in the `StorageFeeEscrow` contract. The automatic tier recalculation algorithm works as follows: (1) calculate the user's new total gross weight for the vault/asset type/user combination, (2) iterate through `tierThresholds` to identify the new tier, (3) if tier has changed, update `currentTierIndex` and recalculate the fee rate, (4) mark `lastRecalculationTime`, and (5) append the fee calculation to `userFeeHistory`. This ensures that users benefit from volume discounts without manual intervention.

I. Risk-Based KYC Expiration and Renewal

KYC credentials in the system are not perpetual; they expire based on assessed risk level. A `KYCProof` with `riskLevel = "low"` is valid for three years; `"medium"` for two years; `"high"` for one year.

Expiration is calculated as: $\text{expirationTime} = \text{issuedAt} + \text{riskLevelDuration}$. Users receive a 30-day grace period after expiration ($\text{graceEndTime} = \text{expirationTime} + 30 \text{ days}$) during which they may not initiate certain actions (claim, collateralize) but may renew their KYC.

After the grace period ends, automated restrictions prevent non-compliant users from executing protected actions. Users may initiate a KYC renewal request, which requires proof of identity and source of funds. Upon approval by the KYC issuer, a new KYCProof is created with an updated expirationTime. The renewal workflow is designed to maintain compliance without creating permanent access restrictions.

J. Information Request Workflow

The Information Request workflow provides structured off-chain communication with on-chain state tracking. An InformationRequest is a template storing: a requestId, a requestType (KYC_RENEWAL, CONSIGNMENT_APPROVAL, DEPOSIT_DETAILS, CLAIM_DOCUMENTATION, etc.), a requester (user or vault), status (PENDING, ACKNOWLEDGED, RESPONDED, COMPLETED, CANCELLED), submittedAt, and off-chain communication metadata (e.g., message hash, IPFS CID). The vault may acknowledge receipt, respond to the request with required information, and mark it as completed.

By default, information requests have a seven-day timeout. If not completed within seven days, the request automatically expires. The vault or the user may cancel the request at any time, triggering state cleanup and any necessary compensating transactions (e.g., releasing locked assets). This workflow reduces friction in off-chain processes while maintaining transparent on-chain records.

K. Atomic Legal Forfeiture Execution

When a court order or regulatory action requires confiscation of a bar, the system supports atomic legal forfeiture. The vault submits a ForfeitureRequest containing: (1) an encrypted court order using ECIES-AES256 encryption, (2) identifiers of the bars to forfeit, (3) a hash of the order for verification, and (4) a cryptographic proof of vault authorization. The Guardian system validates the request via multi-signature approval. Upon approval, the system atomically: (1) transfers ownership of the bars from the user to a custodian account, (2) burns any USG tokens backed by the forfeited bars (preserving the 1:1 backing invariant), and (3) updates the Merkle tree to exclude the bars.

The encrypted court order is stored on-chain but encrypted with the vault's public key, ensuring that only the vault (and authorized parties with access to the private key) can decrypt and verify the order. This design enables enforcement while minimizing disclosure of sensitive legal documents on a transparent ledger.

L. Deposit/Consignment Separation

The Gold Bars system distinguishes between two workflows: (1) deposit, where a user or third party physically delivers bars to the vault and authorizes immediate on-chain custody transfer, and (2) consignment, where a user retains off-chain ownership while the vault stores the bars. A DepositRequest includes the bars, the vault, and immediate consent to custody transfer. A ConsignmentRequest includes the bars, the vault, and a notification to the vault; the vault must explicitly approve for custody to transfer on-chain. Separate request types and state machines ensure that each workflow is handled appropriately.

IV. CONDITIONAL CLAIM PROTOCOL (CCP)

A. Asset Class Registry

The Conditional Claim Protocol includes a dynamic AssetClassRegistry contract enabling registration of new asset types without modifying the core protocol. Each registered asset class includes: (1) a className identifier (e.g., "gold_bars", "real_estate", "fine_art"), (2) a metadataSchema describing available metadata fields and types, (3) a unitCalculationFormula specifying how units are computed from base quantities and quality factors, (4) a baseQuantityUnit (e.g., "troy_ounces" for gold, "square_feet" for real estate), (5) a qualityFactorFormat (e.g., "basis_points" for purity, "grade_scale" for art), and (6) a divisor (typically 10,000 for percentage-based calculations).

Adding a new asset class requires Guardian approval and must specify all required configuration parameters. Upon approval, the asset class becomes available for asset contract instantiation. The registry is immutable once an asset class is registered; modifications require registration of a new asset class version.

B. Linear Unit-Value Calculation

The unit-value calculation formula for all asset classes is linear: $\text{unitValue} = \text{baseQuantity} \times \text{qualityFactor} / \text{divisor}$. This formula is chosen to reduce attack surface compared to more complex alternatives such as scriptable formulas or oracle-driven valuations. For precious metals, baseQuantity is grossWeightOzt, qualityFactor is purity, and divisor is 10,000, yielding the same fine ounces calculation as described in Section III.B.

The linear formula is transparent, deterministic, and can be verified independently without trust in external systems. Future versions of the protocol may support piecewise linear formulas (different slopes in different ranges) or oracle-driven valuations for asset classes requiring dynamic pricing; these are designed as future extensions preserving backwards compatibility.

C. Shared Protocol Components

The CCP shares several key components across all asset classes, reducing development burden and ensuring consistent governance and compliance. These shared components include: (1) Merkle-based proof-of-reserves, with leaf structure adapted for each asset class but using the same SHA-256 hashing and verification algorithms; (2) reusable KYC/compliance proofs with risk-based expiration, described in Section III.I, applicable to all asset classes; (3) prepaid tiered storage fee escrow, with fee rates and thresholds customizable per asset class but using the same percentage-based calculation mechanism; and (4) Guardian multi-party approval, inherited from the Guardian system, for governance and high-risk actions.

Sharing these components eliminates redundancy, reduces bugs through code reuse, ensures consistent governance policies across all asset classes, and enables future enhancements (e.g., cross-collateral fee calculation) that would be difficult if each asset class had independent implementations.

D. Conditional Claim Instruments

A key innovation of the CCP is the conditional claim instrument, which represents a right to obtain full ownership and control of a physical asset upon satisfaction of specified conditions, rather than a custodial IOU conferring possession but retaining issuer liability. Conditions typically include:

(1) presentation of the AssetContract (proof of rightful USG/token holder), (2) payment of outstanding fees and taxes, (3) passing KYC/AML/sanctions checks, and (4) compliance with all applicable jurisdictional regulations.

Prior to claiming an asset (e.g., Gold Bar NFT), the user is KYC'd and checked against AML and sanctions lists. Successful KYC clearance and sanctions check verification confirm that the user is eligible to own the underlying asset. Ownership of the AssetContract itself is verification of ability to own the underlying asset, enabling atomic transfer of custody upon claim without requiring repeated KYC reviews for each transaction. This design simplifies the user experience while maintaining regulatory compliance.

E. Fractionalization

The CCP includes a `supportsFractionalization` flag in each asset class configuration, enabling future development of sub-claim structures. Fractionalization would allow splitting a single asset contract into multiple sub-claims, each representing a fractional ownership or control right over portions of the underlying asset. For example, a large real estate property might be fractionalized into parcels; a fine art collection might be split among multiple collectors.

No full implementation plan for fractionalization is currently specified in this application; the flag is included as forward-looking design that permits future development without requiring protocol changes. Asset classes created with `supportsFractionalization = false` are guaranteed not to be fractionalizable; classes created with `supportsFractionalization = true` may support fractionalization in future versions of the protocol.

F. Deposit, Consignment, and Information Requests

The CCP generalizes the deposit/consignment and information request workflows of the Gold Bars system to all asset classes. A `DepositRequest` specifies an asset contract, custodian, and immediate consent to on-chain custody transfer. A `ConsignmentRequest` specifies an asset contract, custodian, and retains off-chain ownership until explicit approval. An `InformationRequest` provides a template for any custodian-to-user or user-to-custodian communication requiring on-chain tracking.

All information requests have a default seven-day timeout. Either the custodian or the user may cancel the request at any time, triggering automatic cleanup and compensating transactions. This uniform workflow design enables consistent user experience and operational procedures across all supported asset classes.

V. INTEGRATION OF GUARDIAN, GOLD BARS, AND CCP

The three systems described above are designed to operate as an integrated platform. The Guardian library serves as the governance control plane, inherited by both Gold Bars and CCP contracts. Guardian-governed actions include vault/custodian approvals, forfeiture execution, emergency controls, and proof-of-reserves updates. Owner-only actions -- collateralize, uncollateralize, claim, transfer -- execute atomically without Guardian approval to avoid queue-based latency.

Shared infrastructure includes: (i) Merkle-based proof-of-reserves updated atomically with mint/burn operations; (ii) reusable `KYCProof/ComplianceProof` contracts storing non-personally-identifying metadata with risk-based expiration; (iii) prepaid tiered storage fee escrow

calculated as a percentage of base quantity to eliminate oracle dependence; and (iv) Guardian multi-party approval for governance and high-risk actions.

The Gold Bars system is a specialized, production-ready implementation of the CCP framework optimized for precious metals. The CCP extends this foundation to arbitrary asset classes while preserving the same mathematical guarantees. Together, the three systems provide a comprehensive platform for institutional-grade real-world asset tokenization with programmable governance, provable backing, and automated compliance.

BRIEF DESCRIPTION OF THE DRAWINGS

The drawings are described in the accompanying Brief Description of the Drawings document and Patent Figures, which are incorporated herein by reference.

U.S. Patent Application Publication No. 2025/0086712, entitled "Distributed Ledger Tokenization System," is hereby incorporated by reference in its entirety.

ABSTRACT

A system and method for distributed ledger governance, tokenization, and multi-asset registry management. A first aspect provides an inheritable Guardian Multi-Signature Approval System with symmetric approval/disapproval voting, deadlock prevention via the formula $\text{disapprovalThreshold} = \min(\text{approvalThreshold}, \text{guardianCount} - \text{approvalThreshold} + 1)$, sequential execution enforcement, and dynamic threshold management. A second aspect provides a Precious Metal Tokenization System with mathematically enforced 1:1 backing ($\text{totalUSGSupply} == \text{sum of fine metal ounces in collateralized contracts}$), privacy-preserving Merkle proof-of-reserves, tiered storage fees with automatic tier recalculation, risk-based KYC expiration, and atomic legal forfeiture. A third aspect provides a Conditional Claim Protocol with a configurable asset class registry, linear unit-value calculation, conditional claim instruments, and shared compliance/governance/fee infrastructure. The three systems integrate to deliver institutional-grade real-world asset tokenization with programmable governance, provable backing, and automated compliance.

BRIEF DESCRIPTION OF THE DRAWINGS

The following is a brief description of the drawings, which are not necessarily drawn to scale. For each drawing, an overview figure (designated "A") shows the complete drawing for context, and four detail figures (designated "B" through "E") show enlarged quadrant views of the same drawing to ensure legibility of text and detail. The quadrant views include overlapping margins to preserve continuity of content near quadrant boundaries.

FIG. 1A -- Guardian Use Cases for Institutional Smart Contract Governance (Overview)

FIG. 1A is an overview of the complete drawing. This figure illustrates non-limiting use cases in which a Guardian governance library is embedded into one or more smart contracts to control execution of protected actions requiring multi-party approval. Example use cases include cross-institution approvals, administrative controls, compliance-sensitive state changes, emergency actions, and controlled upgrades, while allowing ordinary owner-only or routine actions to execute normally.

FIG. 1B is an enlarged detail view of the upper-left quadrant of FIG. 1A.

FIG. 1C is an enlarged detail view of the upper-right quadrant of FIG. 1A.

FIG. 1D is an enlarged detail view of the lower-left quadrant of FIG. 1A.

FIG. 1E is an enlarged detail view of the lower-right quadrant of FIG. 1A.

FIG. 2A -- Privacy-Preserving Execution and Visibility Model for Guardian on a Permissioned Ledger (Overview)

FIG. 2A is an overview of the complete drawing. This figure depicts a privacy/visibility model in which request data and approvals are visible only to relevant parties (e.g., participants to the transaction and authorized observers), while non-relevant parties receive restricted or no visibility. The figure further shows a multi-party workflow in which approvals are gathered and, upon threshold satisfaction, an atomic transaction is executed while maintaining party-specific privacy constraints.

FIG. 2B is an enlarged detail view of the upper-left quadrant of FIG. 2A.

FIG. 2C is an enlarged detail view of the upper-right quadrant of FIG. 2A.

FIG. 2D is an enlarged detail view of the lower-left quadrant of FIG. 2A.

FIG. 2E is an enlarged detail view of the lower-right quadrant of FIG. 2A.

FIG. 3A -- Comparison of Embedded Guardian Governance to External or Simple Multi-Signature Controls (Overview)

FIG. 3A is an overview of the complete drawing. This figure compares Guardian's embedded governance model (e.g., inheritable library, sequential approval queue, policy-based visibility, richer state machine, and fail-safe mechanisms) to simpler multi-signature patterns that typically lack deterministic ordering, queue unblocking, request lifecycle management, or granular per-action policies. The comparison highlights functional differences relevant to institutional adoption, auditability, and risk management.

FIG. 3B is an enlarged detail view of the upper-left quadrant of FIG. 3A.

FIG. 3C is an enlarged detail view of the upper-right quadrant of FIG. 3A.

FIG. 3D is an enlarged detail view of the lower-left quadrant of FIG. 3A.

FIG. 3E is an enlarged detail view of the lower-right quadrant of FIG. 3A.

FIG. 4A -- Cross-Institutional Approval and Settlement Workflow Using Guardian (Overview)

FIG. 4A is an overview of the complete drawing. This figure shows an example workflow spanning multiple organizations (e.g., institution A, institution B, custodian/vault, compliance party, observer/auditor), in which a protected action is submitted, routed for approvals according to per-action policy, and executed atomically after threshold satisfaction. The figure emphasizes governance across organizational boundaries without a single controlling party.

FIG. 4B is an enlarged detail view of the upper-left quadrant of FIG. 4A.

FIG. 4C is an enlarged detail view of the upper-right quadrant of FIG. 4A.

FIG. 4D is an enlarged detail view of the lower-left quadrant of FIG. 4A.

FIG. 4E is an enlarged detail view of the lower-right quadrant of FIG. 4A.

FIG. 5A -- Value Proposition and Integration Overview for Guardian Governance (Overview)

FIG. 5A is an overview of the complete drawing. This figure presents an overview of Guardian's value proposition, including ordered execution, privacy-aware approvals, audit trail generation, and fail-safe recovery paths such as cancellation or expiration. The figure also illustrates how Guardian integrates as a reusable component within application smart contracts to enforce governance policies while preserving efficient execution for non-governed actions.

FIG. 5B is an enlarged detail view of the upper-left quadrant of FIG. 5A.

FIG. 5C is an enlarged detail view of the upper-right quadrant of FIG. 5A.

FIG. 5D is an enlarged detail view of the lower-left quadrant of FIG. 5A.

FIG. 5E is an enlarged detail view of the lower-right quadrant of FIG. 5A.

FIG. 6A -- Asset Lifecycle State Machine for Tokenized Physical Assets (Overview)

FIG. 6A is an overview of the complete drawing. This figure shows a non-limiting state machine for an asset contract representing a physical asset (e.g., a gold bar), including states associated with deposit/consignment initiation, review outcomes (approved/denied/information request), custody placement, collateralization status, claim/redemption workflows, and terminal outcomes. The state machine provides an auditable on-chain record of asset status transitions.

FIG. 6B is an enlarged detail view of the upper-left quadrant of FIG. 6A.

FIG. 6C is an enlarged detail view of the upper-right quadrant of FIG. 6A.

FIG. 6D is an enlarged detail view of the lower-left quadrant of FIG. 6A.

FIG. 6E is an enlarged detail view of the lower-right quadrant of FIG. 6A.

FIG. 7A -- Public Claim Enablement States and Forced-On Claimability While Collateralized (Overview)

FIG. 7A is an overview of the complete drawing. This figure depicts modes in which public claimability may be disabled or enabled when an asset is uncollateralized, and a forced-on mode in which public claimability is enabled and non-disableable while the asset is collateralized to support circulating tokens. The figure illustrates how claimability settings affect who may claim an asset contract, subject to eligibility requirements (e.g., compliance proofs).

FIG. 7B is an enlarged detail view of the upper-left quadrant of FIG. 7A.

FIG. 7C is an enlarged detail view of the upper-right quadrant of FIG. 7A.

FIG. 7D is an enlarged detail view of the lower-left quadrant of FIG. 7A.

FIG. 7E is an enlarged detail view of the lower-right quadrant of FIG. 7A.

FIG. 8A -- Compliance Proof Validity, Expiration, Warning/Grace Periods, and Action Gating (Overview)

FIG. 8A is an overview of the complete drawing. This figure illustrates a compliance proof (e.g., KYCProof/ComplianceProof) lifecycle including valid, warning, grace, and expired states based on an expiration timestamp and optionally risk level. The figure further shows gating rules distinguishing actions that remain permitted (e.g., viewing balances or proofs) from actions that become restricted (e.g., title-changing actions such as claim, redemption, or certain custody operations) once a proof expires beyond a grace period.

FIG. 8B is an enlarged detail view of the upper-left quadrant of FIG. 8A.

FIG. 8C is an enlarged detail view of the upper-right quadrant of FIG. 8A.

FIG. 8D is an enlarged detail view of the lower-left quadrant of FIG. 8A.

FIG. 8E is an enlarged detail view of the lower-right quadrant of FIG. 8A.

FIG. 9A -- Legal Forfeiture / Forced Redemption Workflow with Multi-Party Authorization and Backing Preservation (Overview)

FIG. 9A is an overview of the complete drawing. This figure depicts a forced redemption process triggered by a legal directive (e.g., court order), including storing a document hash and encrypted reference, multi-party verification/authorization (e.g., via Guardian), and an execution stage that force-transfers an asset contract to a recipient. In collateralized cases, the workflow preserves a backing invariant by burning tokens and updating proof-of-reserves commitments as part of the forfeiture execution.

FIG. 9B is an enlarged detail view of the upper-left quadrant of FIG. 9A.

FIG. 9C is an enlarged detail view of the upper-right quadrant of FIG. 9A.

FIG. 9D is an enlarged detail view of the lower-left quadrant of FIG. 9A.

FIG. 9E is an enlarged detail view of the lower-right quadrant of FIG. 9A.

FIG. 10A -- Sequential Atomic Collateralization Operation with Proof-of-Reserves Update (Overview)

FIG. 10A is an overview of the complete drawing. This figure shows an example atomic sequence for collateralizing an asset contract, including changing the asset contract to a collateralized

state, minting a corresponding amount of fungible tokens, and updating a proof-of-reserves commitment (e.g., adding a Merkle leaf and updating a Merkle root) within a single transaction. The figure emphasizes consistency guarantees achieved through atomic execution.

FIG. 10B is an enlarged detail view of the upper-left quadrant of FIG. 10A.

FIG. 10C is an enlarged detail view of the upper-right quadrant of FIG. 10A.

FIG. 10D is an enlarged detail view of the lower-left quadrant of FIG. 10A.

FIG. 10E is an enlarged detail view of the lower-right quadrant of FIG. 10A.

FIG. 11A -- Sequential Atomic Claim Operation Including Burn and Proof-of-Reserves Leaf Removal (Overview)

FIG. 11A is an overview of the complete drawing. This figure depicts an example atomic sequence for claiming a collateralized asset, including verifying claimant eligibility, burning fungible tokens (or otherwise settling consideration), removing the corresponding proof-of-reserves leaf (and updating the root), and transferring the asset contract to the claimant within a single transaction. The figure emphasizes that claim transitions preserve backing invariants and auditable state changes.

FIG. 11B is an enlarged detail view of the upper-left quadrant of FIG. 11A.

FIG. 11C is an enlarged detail view of the upper-right quadrant of FIG. 11A.

FIG. 11D is an enlarged detail view of the lower-left quadrant of FIG. 11A.

FIG. 11E is an enlarged detail view of the lower-right quadrant of FIG. 11A.

FIG. 12A -- Guardian Sequential Approval Queue and Request Lifecycle States (Overview)

FIG. 12A is an overview of the complete drawing. This figure depicts a sequential approval queue in which protected action requests are assigned identifiers/positions and transition among lifecycle states such as submitted, awaiting approvals, threshold met, executed, expired, or canceled. The figure further illustrates queue unblocking mechanisms including timeout expiration and cancellation (including out-of-order cancellation in some embodiments), enabling deterministic processing and prevention of indefinite queue blockage.

FIG. 12B is an enlarged detail view of the upper-left quadrant of FIG. 12A.

FIG. 12C is an enlarged detail view of the upper-right quadrant of FIG. 12A.

FIG. 12D is an enlarged detail view of the lower-left quadrant of FIG. 12A.

FIG. 12E is an enlarged detail view of the lower-right quadrant of FIG. 12A.

FIG. 13A -- Multi-Asset Tokenization via Asset Class Registry and Linear Unit-Value Semantics (Overview)

FIG. 13A is an overview of the complete drawing. This figure compares example asset classes supported by a Conditional Claim Protocol, such as precious metals and real estate, and shows how an AssetClassRegistry may define base quantity units, quality factor formats, divisors, and linear unit-value formulas per class. The figure also highlights shared protocol components (e.g., proof-of-reserves, compliance proofs, fee escrow, and governance controls) that operate consistently across asset classes.

FIG. 13B is an enlarged detail view of the upper-left quadrant of FIG. 13A.

FIG. 13C is an enlarged detail view of the upper-right quadrant of FIG. 13A.

FIG. 13D is an enlarged detail view of the lower-left quadrant of FIG. 13A.

FIG. 13E is an enlarged detail view of the lower-right quadrant of FIG. 13A.

FIG. 14A -- Guardian Library Architecture, Action Partitioning, and Reuse Across Applications (Overview)

FIG. 14A is an overview of the complete drawing. This figure illustrates the architecture of a Guardian library that is inherited, imported, or composed into an application contract. The figure distinguishes Guardian-governed actions (e.g., administrative or high-risk state changes) from owner-only actions that execute atomically without entering the Guardian queue. The figure also depicts how shared components (e.g., compliance proofs and proof-of-reserves updates) may interface with governance policies.

FIG. 14B is an enlarged detail view of the upper-left quadrant of FIG. 14A.

FIG. 14C is an enlarged detail view of the upper-right quadrant of FIG. 14A.

FIG. 14D is an enlarged detail view of the lower-left quadrant of FIG. 14A.

FIG. 14E is an enlarged detail view of the lower-right quadrant of FIG. 14A.

PROVISIONAL PATENT APPLICATION

GUARDIAN MULTI-SIGNATURE APPROVAL SYSTEM FOR DISTRIBUTED LEDGER TECHNOLOGY

Title: Inheritable Multi-Signature Governance System with Sequential Execution and Deadlock Prevention for Distributed Ledger Technology

Inventor(s): [To be filled]

Assignee: [To be filled]

Date: [To be filled]

Related Application: This application references U.S. Patent Application No. 2025/0086712 (SGU Patent) for context on distributed ledger implementations.

ABSTRACT

A system and method for embedding multi-signature governance directly into distributed ledger smart contracts through an inheritable template pattern. The system provides configurable multi-party authorization with dynamic threshold management, strict sequential execution with configurable out-of-order exceptions, symmetric approval/disapproval voting with mathematical deadlock prevention, and automatic request expiration handling. The system is designed for privacy-preserving distributed ledger networks such as Canton Network using Daml, but is applicable to any distributed ledger technology including Ethereum Virtual Machine (EVM) compatible blockchains.

FIELD OF THE INVENTION

This invention relates to distributed ledger technology, smart contract security, and decentralized governance systems. More specifically, it relates to multi-signature approval systems embedded directly within smart contract code rather than implemented as external wallet systems.

BACKGROUND OF THE INVENTION

Traditional multi-signature (multisig) systems for blockchain applications typically use external wallet contracts (such as Gnosis Safe) that control other contracts from outside. This creates several limitations:

1. **External Dependency:** Contracts must trust external multisig wallets, creating a single point of failure
2. **No Sequential Execution:** Multiple transactions can execute simultaneously, creating race conditions
3. **No Bidirectional Voting:** Traditional multisig systems only support approval, not disapproval
4. **Deadlock Risk:** Without proper threshold calculation, systems can deadlock where neither approval nor disapproval can proceed

5. **No Per-Contract Independence:** All contracts share the same multisig wallet, reducing flexibility

The present invention addresses these limitations by embedding governance logic directly into contract code at compile time, enabling each contract to have its own independent guardian system with advanced features including sequential execution enforcement and mathematical deadlock prevention.

SUMMARY OF THE INVENTION

The Guardian Multi-Signature Approval System is an inheritable, reusable framework for distributed ledger technology that provides:

1. **Inheritable Template Pattern:** Enables any contract to gain multi-sig capabilities through inheritance/composition
 2. **Symmetric Approval/Disapproval Voting:** Guardians can approve OR disapprove requests with intelligent threshold calculation
 3. **Sequential Execution Enforcement:** Requests execute in strict order (by request ID) to prevent race conditions, with configurable out-of-order exceptions for emergency actions
 4. **Mathematical Deadlock Prevention:** Guaranteed impossibility of simultaneous approval and disapproval threshold achievement
 5. **Dynamic Threshold Management:** Automatic threshold adjustment when guardian sets change
 6. **Progressive Expiration Cleanup:** Automatic cleanup of expired requests to maintain queue health
 7. **Action-Specific Configuration:** Different thresholds and guardian sets per action type
-

DETAILED DESCRIPTION OF THE INVENTION

System Architecture

Core Data Structures

The system uses the following core data structures:

PendingRequest/PendingAction Template:

- **requestId:** Unique sequential identifier
- **contractCid:** Reference to parent contract
- **actionType:** Type of action being requested
- **submitter:** Party/address that submitted the request
- **submittedAt:** Timestamp of submission
- **approvals:** Set of guardians who have approved
- **disapprovals:** Set of guardians who have disapproved
- **finalized:** Boolean indicating if request has been executed

- `uintParams`, `addressParams`, `bytes32Params`: Action-specific parameters

Guardian Configuration:

- `guardians`: List of guardian parties/addresses
- `defaultThreshold`: Global threshold for all actions
- `actionThresholds`: Map of action-specific thresholds
- `requestTimeout`: Time duration after which requests expire
- `nextRequestId`: Counter for next request ID
- `nextExecutableRequestId`: Pointer to next request that can execute

Inheritance Pattern

The system uses an abstract base template/contract that child contracts inherit from:

Daml Implementation:

```
class HasGuardian a where
  getGuardianConfig : a -> GuardianConfig
  executeAction : a -> Text -> [Int] -> [Party] -> [Text] -> Update (ContractId a)
```

Solidity Implementation:

```
abstract contract Guardian {
  // Core functionality implemented here
  // Child contracts inherit and gain guardian capabilities
}
```

Core Algorithms

1. Action Submission Workflow

When an action is submitted:

1. **Input Validation**: Verify action type and parameters
2. **Expired Request Cleanup**: Query and archive expired requests (bounded cleanup)
3. **Parameter Validation**: Validate parameters based on action type
4. **Request Creation**: Create new PendingRequest contract with:
 - Incremented `nextRequestId`
 - Empty approval/disapproval sets
 - Current timestamp
 - Action parameters
5. **Config Update**: Update GuardianConfig with new `nextRequestId`
6. **Immediate Execution Check**: If threshold is 0 and request is next executable, execute immediately

2. Approval Workflow

When a guardian approves a request:

1. **Validation:** Verify request exists, not finalized, guardian hasn't voted, guardian $\neq$ submitter
2. **State Update:** Add guardian to approvals set (via contract archival/creation in Daml, or state update in Solidity)
3. **Threshold Check:** Calculate new approval count
4. **Execution Decision:**
 - If `approvalCount >= threshold` AND (`requestId == nextExecutableRequestId` OR `canExecuteOutOfOrder(actionType)`):
 - Execute action immediately
 - Else:
 - Wait in queue for sequential execution

3. Disapproval Workflow

When a guardian disapproves a request:

1. **Validation:** Verify request exists, not finalized, guardian hasn't voted, guardian $\neq$ submitter
2. **State Update:** Add guardian to disapprovals set
3. **Disapproval Threshold Check:** Calculate new disapproval count
4. **Cancellation:** If `disapprovalCount >= disapprovalThreshold`:
 - Cancel request immediately (out-of-order execution allowed for disapproval)
 - Archive request contract
 - Update queue pointer if needed

THE DEADLOCK FORMULA

Critical Innovation: The system uses a mathematical formula to calculate the disapproval threshold that guarantees deadlock prevention.

Formula:

$$\text{disapprovalThreshold} = \min(\text{approvalThreshold}, \text{guardianCount} - \text{approvalThreshold} + 1)$$

Mathematical Proof of Deadlock Prevention:

Let:

- G = total number of guardians
- A = approval threshold
- D = disapproval threshold = $\min(A, G - A + 1)$

Case 1: If $A \leq G - A + 1$, then $D = A$

- Maximum possible approvals: A

- Maximum possible disapprovals: A
- Since guardians cannot vote both approve and disapprove: $\text{approvals} + \text{disapprovals} \leq G$
- If both thresholds met: $A + A \leq G \rightarrow 2A \leq G$
- But $D = A$ and $A \leq G - A + 1 \rightarrow 2A \leq G + 1$
- For deadlock: need A approvals AND A disapprovals = $2A$ votes
- Since $2A \leq G + 1$ and guardians are distinct, this requires at least one guardian voting twice, which is impossible.

Case 2: If $A > G - A + 1$, then $D = G - A + 1$

- Maximum possible approvals: A
- Maximum possible disapprovals: $G - A + 1$
- If both thresholds met: $A + (G - A + 1) = G + 1$ votes required
- But only G guardians exist, so this is mathematically impossible.

Conclusion: The formula $D = \min(A, G - A + 1)$ guarantees that $\text{approvalCount} + \text{disapprovalCount} > G$ whenever both thresholds are met, which is impossible since there are only G guardians and each guardian can vote at most once.

Inequality Guarantee:

$$\text{approvalThreshold} + \text{disapprovalThreshold} > \text{guardianCount}$$

This inequality ensures that it is mathematically impossible for both approval and disapproval thresholds to be met simultaneously, preventing deadlocks.

OUT-OF-ORDER EXECUTION EXCEPTION

Primary Claim: Strict Sequential Execution - requests execute in order by request ID ($\text{requestId} == \text{nextExecutableRequestId}$).

Exception Conditions: The system allows out-of-order execution ONLY under the following specific conditions:

1. **Emergency Actions:** Action types that are designated as emergency actions can execute out-of-order when approval threshold is met, regardless of request ID. These include:
 - **VAULT_LOCK:** Lock the vault (emergency security measure)
 - **VAULT_UNLOCK:** Unlock the vault (emergency recovery)
 - **VAULT_RECOVER:** Vault recovery actions (emergency access restoration)
 - **VAULT_EXECUTE_RECOVERY:** Execute vault recovery (emergency execution)
2. **Disapproval Cancellation:** When disapproval threshold is met, the request is cancelled immediately (out-of-order) regardless of request ID. This prevents malicious or erroneous requests from blocking the queue.

Implementation Logic:

```
-- Pseudo-code

canExecuteOutOfOrder : Text -> Bool
canExecuteOutOfOrder actionType =
  actionType == "VAULT_LOCK" ||
  actionType == "VAULT_UNLOCK" ||
  actionType == "VAULT_RECOVER" ||
  actionType == "VAULT_EXECUTE_RECOVERY"

-- In approval workflow:
IF newApprovalCount >= threshold:
  IF requestId == nextExecutableRequestId OR canExecuteOutOfOrder(actionType):
    Call finalizeAction(pendingActionCid)
  ELSE:
    -- Wait in queue (strict sequential execution)
    Return
```

Scope Limitation: The out-of-order exception is narrowly scoped to:

- Only emergency security/recovery actions
- Only when approval threshold is met (for emergency actions)
- Only when disapproval threshold is met (for cancellation)
- All other actions strictly follow sequential execution

This exception does not invalidate the primary claim of sequential execution because:

1. The exception is explicitly defined and limited
2. The exception serves a specific security purpose (emergency response)
3. The sequential execution rule applies to all non-emergency actions
4. The system maintains deterministic execution order for all standard operations

THRESHOLD DYNAMICS

Question: When a guardian is added or removed, does the threshold recalculate immediately in the same block, or is there a delay?

Answer: Immediate Recalculation in the Same Transaction/Block

Implementation Details:

1. Guardian Addition/Removal Transaction:

- Guardian add/remove actions themselves require guardian approval (via the Guardian system)
- When approved, the transaction atomically: a. Updates the guardian set (adds/removes guardian) b. Recalculates all thresholds based on new guardian

count c. Updates GuardianConfig contract with new thresholds d. All subsequent requests use the new thresholds immediately

2. Threshold Recalculation Formula:

- For each action type with threshold T and guardian count G :
 - If threshold is percentage-based: $\text{newThreshold} = \text{ceiling}(T * \text{newGuardianCount} / \text{oldGuardianCount})$
 - If threshold is absolute: $\text{newThreshold} = \text{min}(T, \text{newGuardianCount})$ (cannot exceed guardian count)
 - Disapproval threshold recalculated using deadlock formula: $D = \text{min}(A, G - A + 1)$
3. **Atomicity:** In Canton/Daml, all state changes in a transaction are atomic. The guardian set update and threshold recalculation occur in the same atomic transaction, ensuring:
- No intermediate state where guardian count and thresholds are inconsistent
 - Immediate effect: next request submission uses updated thresholds
 - No delay or separate transaction needed
4. **Pending Requests:** Requests that are already pending use the thresholds that were in effect at the time of submission. Only new requests use the updated thresholds.

Example:

- Initial state: 5 guardians, approval threshold = 3
- Guardian removal approved: 4 guardians remain
- Same transaction: Threshold recalculated to maintain same security level (e.g., 3-of-4 or adjusted proportionally)
- Next request submission: Uses new threshold immediately

DRAWINGS

FIG. 1: System Architecture

Description:

FIG. 1 illustrates the system architecture showing the inheritance relationship between the GuardianBase contract/template and a TargetContract that inherits from it.

Elements:

- **GuardianBase** (left box): Contains core guardian functionality including:
 - Guardian configuration management
 - Request queue management
 - Approval/disapproval voting logic
 - Threshold calculation algorithms
 - Sequential execution enforcement

- **TargetContract** (right box): Inherits from GuardianBase and contains:
 - Contract-specific business logic
 - Action execution implementations
 - Custom action validation
- **Inheritance Arrow**: Solid arrow pointing from TargetContract to GuardianBase, labeled "inherits from" or "extends"
- **Guardian Configuration**: Shown as shared data structure accessible to both
- **Pending Requests Queue**: Shown as data structure managed by GuardianBase

Layout: Two boxes side by side, GuardianBase on left, TargetContract on right, with inheritance arrow connecting them. Below, show shared data structures (GuardianConfig, PendingRequests queue).

FIG. 2: Request Lifecycle Flowchart

Description:

FIG. 2 is a flowchart showing the complete lifecycle of a request from submission through finalization or expiration.

Flow Steps:

1. **START** (top, rounded rectangle)
2. **Submit Action Request** (rectangle)
 - Arrow down to:
3. **Validate Inputs** (diamond decision)
 - "Valid?" → Yes: continue down
 - "Valid?" → No: arrow to **REJECT** (terminator, left side)
4. **Create PendingRequest** (rectangle)
 - Set requestId, timestamp, empty approvals/disapprovals
 - Arrow down to:
5. **Add to Queue** (rectangle)
 - Arrow down to:
6. **Guardians Vote** (rectangle, parallel process indicator)
 - Arrow splits to two paths:
 - **Path A (Approval)**: Arrow right to **Guardian Approves** → **Update Approval Count**
 - **Path B (Disapproval)**: Arrow right to **Guardian Disapproves** → **Update Disapproval Count**
7. **Check Thresholds** (diamond decision, center)
 - Three decision branches:
 - **Approval Threshold Met?** → Yes: arrow down to **Check Execution Conditions**
 - **Disapproval Threshold Met?** → Yes: arrow down to **Cancel Request** (terminator, left)
 - **Neither Met?** → Arrow looping back up to **Guardians Vote**
8. **Check Execution Conditions** (diamond decision)
 - **requestId == nextExecutableRequestId OR Emergency Action?** → Yes: arrow down to **Execute Action**

- **No:** Arrow down to **Wait in Queue** (rectangle) → Arrow looping back to **Check Thresholds**
- 9. **Execute Action** (rectangle)
 - Arrow down to:
- 10. **Finalize Request** (rectangle)
 - Arrow down to:
- 11. **END** (bottom, rounded rectangle)

Additional Paths:

- From **Add to Queue**: Arrow branching to **Check Expiration** (diamond)
 - **Expired?** → Yes: arrow to **Archive Expired Request** (terminator, right)
 - **Not Expired?** → Arrow to **Guardians Vote**

Layout: Top-to-bottom flowchart with START at top, main flow down center, rejection/cancellation paths on left, expiration path on right.

FIG. 3: Voting Logic Flowchart

Description:

FIG. 3 is a detailed flowchart showing the voting logic sequence: Check Vote → Update Count → Check Deadlock Algorithm.

Flow Steps:

1. **START** (top, rounded rectangle)
2. **Guardian Submits Vote** (rectangle)
 - Arrow down to:
3. **Validate Guardian** (diamond decision)
 - **Is Guardian?** → No: arrow to **REJECT** (terminator, left)
 - **Is Guardian?** → Yes: arrow down to:
4. **Check Vote Type** (diamond decision)
 - **Approve?** → Arrow right to **Approval Path**
 - **Disapprove?** → Arrow down to **Disapproval Path**
5. **Approval Path:**
 - **Check Existing Votes** (diamond)
 - **Already Voted?** → Yes: arrow to **REJECT** (terminator)
 - **Already Voted?** → No: arrow down to:
 - **Update Approval Count** (rectangle)
 - `approvalCount = approvalCount + 1`
 - Arrow down to:
 - **Calculate New Approval Count** (rectangle)
 - Arrow down to:
 - **Check Approval Threshold** (diamond)
 - `approvalCount >= threshold?` → No: arrow to **END** (terminator, bottom)
 - `approvalCount >= threshold?` → Yes: arrow down to:

- **Check Execution Conditions** (diamond)
 - **requestId == nextExecutableRequestId OR canExecuteOutOfOrder?**
 - Yes: arrow to **EXECUTE** (terminator, right)
 - **No**: arrow to **QUEUE** (terminator, bottom)
- 6. **Disapproval Path:**
 - **Check Existing Votes** (diamond)
 - **Already Voted?** → Yes: arrow to **REJECT** (terminator)
 - **Already Voted?** → No: arrow down to:
 - **Update Disapproval Count** (rectangle)
 - `disapprovalCount = disapprovalCount + 1`
 - Arrow down to:
 - **Calculate Disapproval Threshold** (rectangle, shows formula)
 - `disapprovalThreshold = min(approvalThreshold, guardianCount - approvalThreshold + 1)`
 - Arrow down to:
 - **Check Disapproval Threshold** (diamond)
 - **disapprovalCount >= disapprovalThreshold?** → No: arrow to **END** (terminator, bottom)
 - **Yes**: arrow down to:
 - **Check Deadlock Prevention** (rectangle, shows mathematical proof)
 - "Verify: approvalCount + disapprovalCount cannot exceed guardianCount"
 - Arrow down to:
 - **CANCEL REQUEST** (terminator, right, emphasized)

Key Elements:

- Formula box showing: `disapprovalThreshold = min(approvalThreshold, guardianCount - approvalThreshold + 1)`
- Deadlock prevention verification step
- Clear separation between approval and disapproval paths

Layout: Top-to-bottom flowchart with START at top, branching to Approval (left) and Disapproval (right) paths, rejoining at threshold checks, with formula and deadlock check prominently displayed.

FIG. 4: Sequential Execution Logic Gate

Description:

FIG. 4 illustrates the logic gate that enforces sequential execution, checking whether `requestId == nextExecutableRequestId`.

Elements:

1. **Input A:** `requestId` (current request's ID) - shown as input on left
2. **Input B:** `nextExecutableRequestId` (queue pointer) - shown as input on left, below Input A

3. **Comparator:** "==" (equals) logic gate in center
 - Receives Input A and Input B
 - Outputs boolean: `requestId == nextExecutableRequestId`
4. **OR Gate:** Receives:
 - Output from Comparator (==)
 - `canExecuteOutOfOrder(actionType)` boolean input
5. **Final Output:** Boolean result determining if execution can proceed
 - **TRUE:** Execute action
 - **FALSE:** Wait in queue

Additional Elements:

- **Queue Pointer Update:** Shown as feedback loop from execution back to `nextExecutableRequestId`
- **Action Type Input:** Shown feeding into `canExecuteOutOfOrder()` function
- **Emergency Flag:** Visual indicator when out-of-order execution is allowed

Logic Expression:

$\text{CanExecute} = (\text{requestId} == \text{nextExecutableRequestId}) \text{ OR } \text{canExecuteOutOfOrder}(\text{actionType})$

Layout:

- Left side: Two inputs (`requestId`, `nextExecutableRequestId`) feeding into comparator
- Center: Comparator (==) and OR gate
- Right side: Output (Execute/Wait decision)
- Below: Queue pointer update mechanism shown as feedback
- Top: Action type input feeding into out-of-order check function

Visual Style: Digital logic diagram with standard logic gate symbols (comparator, OR gate), clear input/output labels, and flow indicators.

CLAIMS

System Claims

1. A multi-signature approval system for distributed ledger technology comprising:
 - An inheritable base template/contract providing guardian functionality
 - A request queue system maintaining sequential request IDs
 - A voting system supporting both approval and disapproval votes
 - A threshold calculation system using the formula: $\text{disapprovalThreshold} = \min(\text{approvalThreshold}, \text{guardianCount} - \text{approvalThreshold} + 1)$
 - A sequential execution enforcement mechanism requiring `requestId == nextExecutableRequestId` for execution

- An out-of-order execution exception mechanism for emergency actions
2. The system of claim 1, wherein the deadlock prevention formula guarantees that `approvalThreshold + disapprovalThreshold > guardianCount`, making simultaneous threshold achievement mathematically impossible.
 3. The system of claim 1, wherein thresholds recalculate immediately and atomically when guardians are added or removed in the same transaction.
 4. The system of claim 1, wherein out-of-order execution is permitted only for:
 - Emergency action types (lock, unlock, recovery)
 - Disapproval cancellation when disapproval threshold is met

Method Claims

5. A method for preventing deadlocks in a multi-signature voting system comprising:
 - Calculating disapproval threshold using: `disapprovalThreshold = min(approvalThreshold, guardianCount - approvalThreshold + 1)`
 - Ensuring the mathematical guarantee that approval and disapproval thresholds cannot be met simultaneously
6. A method for sequential execution with emergency exceptions comprising:
 - Enforcing strict sequential execution by requiring `requestId == nextExecutableRequestId`
 - Allowing out-of-order execution only when `canExecuteOutOfOrder(actionType)` returns true for emergency actions
 - Maintaining deterministic execution order for all non-emergency actions
7. A method for dynamic threshold management comprising:
 - Immediately recalculating thresholds when guardian set changes
 - Updating thresholds atomically in the same transaction as guardian set changes
 - Applying new thresholds to all subsequent requests immediately

CONCLUSION

The Guardian Multi-Signature Approval System provides a novel approach to embedded governance in distributed ledger technology, with key innovations including:

1. Mathematical deadlock prevention through intelligent threshold calculation
2. Strict sequential execution with narrowly-scoped emergency exceptions
3. Immediate threshold recalculation maintaining security as guardian sets evolve

4. Inheritable design enabling any contract to gain multi-sig capabilities

These innovations address critical limitations in existing multisig systems and provide a robust foundation for secure, decentralized governance on distributed ledgers.

PROVISIONAL PATENT APPLICATION

GOLD BARS V1.3 - DISTRIBUTED LEDGER TOKENIZATION SYSTEM FOR PHYSICAL PRECIOUS METALS

Title: Distributed Ledger Tokenization System with Tiered Storage Fees, Fine Metal Ounces Calculation, and Automated Compliance for Physical Precious Metal Assets

Inventor(s): [To be filled]

Assignee: [To be filled]

Date: [To be filled]

Related Application: This application references U.S. Patent Application No. 2025/0086712 (SGU Patent) for context on distributed ledger implementations, and the Guardian Multi-Signature Approval System for distributed ledger governance.

ABSTRACT

A system and method for tokenizing physical precious metal assets (including but not limited to gold, silver, platinum, and palladium bars) stored in vaults on distributed ledger technology with mathematically enforced 1:1 backing, tiered percentage-based storage fee system with automatic tier recalculation, fine metal ounces calculation supporting multiple purities, risk-based KYC expiration and renewal, and atomic legal forfeiture execution. The system provides per-user prepaid storage periods with asset-type-specific tiered rates, automatic tier boundary crossing detection, and prorated refund calculations. While gold bars are used as the primary example throughout this application, the system applies equally to any precious metal stored in vaults. The system is designed for privacy-preserving distributed ledger networks such as Canton Network using Daml, but is applicable to any distributed ledger technology including Ethereum Virtual Machine (EVM) compatible blockchains.

FIELD OF THE INVENTION

This invention relates to distributed ledger technology, asset tokenization, and commodity-backed digital assets. More specifically, it relates to systems and methods for tokenizing physical precious metal assets (including gold, silver, platinum, palladium, and other precious metals) stored in vaults with mathematically enforced backing ratios, automated storage fee management, and compliance controls. While gold bars are used as the primary example, the invention applies to any precious metal stored in vaults.

BACKGROUND OF THE INVENTION

Traditional systems for tokenizing physical precious metals (such as gold, silver, platinum, and palladium) stored in vaults face several limitations:

1. **No Mathematical Backing Guarantee:** Existing systems cannot mathematically enforce that token supply equals backing assets at all times

2. **Inefficient Storage Fee Models:** Storage fees are typically flat-rate or per-asset, not accounting for volume discounts or asset-type differences
3. **No Automatic Tier Recalculation:** When holdings change, storage fee tiers must be manually recalculated, creating opportunities for errors
4. **Limited Purity Support:** Systems typically support only one purity standard per metal type, limiting flexibility across different precious metals and purities
5. **No KYC Expiration Management:** KYC credentials remain valid indefinitely, creating compliance risks
6. **No Atomic Legal Forfeiture:** Court orders and regulatory actions cannot be executed atomically while preserving backing invariants
7. **No Information Request Workflow:** Deposit/consignment processes lack structured off-chain communication with on-chain state tracking

The present invention addresses these limitations by providing a comprehensive tokenization system with mathematically enforced backing, automated tier recalculation, multi-purity support, risk-based KYC expiration, and atomic legal forfeiture execution.

SUMMARY OF THE INVENTION

The Precious Metal Tokenization System (exemplified by Gold Bars v1.3) is a distributed ledger-based framework that provides:

1. **Mathematically Enforced 1:1 Backing:** Total USG supply equals exact sum of fine metal ounces in all collateralized precious metal bar contracts at all times (exemplified by GoldBar contracts for gold)
2. **Fine Metal Ounces Calculation:** Supports multiple purities for any precious metal (e.g., gold: 99.99%, 99.95%, 99.5%; silver: 99.9%, 99.99%, etc.) using formula: fine metal ounces = gross weight × purity (basis points) / 10,000. While gold bars are used as the primary example, the same calculation applies to silver, platinum, palladium, and other precious metals.
3. **Tiered Percentage-Based Storage Fees:** Asset-type-specific tiered fee structures based on total gross weight holdings per user per asset type per vault
4. **Automatic Tier Recalculation:** When user holdings cross tier boundaries, entire prepaid period recalculated at new tier rate atomically
5. **Per-User Prepaid Periods:** One prepaid period per user per asset type per vault (not per GoldBar), enabling efficient tier management
6. **Risk-Based KYC Expiration:** KYC validity periods (1-3 years) based on risk level, with automated restrictions and renewal workflows
7. **Information Request Workflow:** Off-chain communication with on-chain state transitions for deposit/consignment resolution
8. **Atomic Legal Forfeiture:** Encrypted court order storage with Guardian verification and atomic execution preserving 1:1 backing
9. **Deposit/Consignment Separation:** Distinct flows for physical movement (deposit) and on-chain custody placement (consignment)

DETAILED DESCRIPTION OF THE INVENTION

System Architecture

Core Data Structures

Precious Metal Bar Contract (exemplified as GoldBar Contract):

- **barId**: Unique bar identifier (serial number)
- **vault**: Party managing the vault
- **owner**: Current owner party
- **serialNumber**: Physical bar serial number
- **grossWeightOzt**: Total physical bar weight in troy ounces
- **purity**: Basis points (e.g., 9999 for 99.99%)
- **assetType**: Asset type classification (gold bullion, silver bullion, etc.)
- **collateralized**: Boolean indicating if bar backs USG tokens
- **publicClaimEnabled**: Boolean indicating if bar is claimable by any KYC'd party
- **ipfsCID**: IPFS content identifier for photos, certificate, metadata
- **legalHash**: SHA-256 hash of custody agreement
- **merkleLeafIndex**: Optional index in Merkle tree if collateralized

StorageFeeEscrow Contract:

- **user**: Party owning the prepaid period
- **vault**: Party managing the vault
- **assetType**: Asset type for this prepaid period
- **totalGrossWeight**: Total weight of all bars of this asset type
- **tier**: Current tier based on total weight
- **lockedInRate**: Rate locked in at prepayment (basis points)
- **prepaidUntil**: Date when prepaid period expires
- **monthlyFeeAmount**: Calculated monthly fee at locked-in rate

KYCProof Contract:

- **user**: Party that completed KYC
- **providerId**: KYC provider identifier (e.g., "sumsub")
- **refId**: Provider reference ID
- **level**: KYC verification level
- **countryCode**: ISO country code
- **expiresAt**: Expiration timestamp
- **riskLevel**: Risk classification (LOW, MEDIUM, HIGH)

Core Algorithms

1. Fine Metal Ounces Calculation

Critical Innovation: The system calculates fine (pure) metal content using a formula that supports all precious metal purities. While gold bars are used as the primary example, the same calculation applies to silver, platinum, palladium, and other precious metals stored in vaults:

$$\text{fineGoldOzt} = \text{grossWeightOzt} \times \text{purity} / 10,000$$

Where:

- **grossWeightOzt** = total physical bar weight in troy ounces
- **purity** = purity in basis points (e.g., for gold: 9999 for 99.99%, 9995 for 99.95%, 9950 for 99.5%; for silver: 9990 for 99.9%, 9999 for 99.99%; similar for platinum, palladium, etc.)
- **fineMetalOzt** = fine metal ounces (pure metal content - gold, silver, platinum, palladium, etc.)

Examples (Gold Bars):

- 100 toz gold bar at 99.99% purity (9999 basis points): $100 \times 9999 / 10,000 = 99.99$ fine ozt
- 100 toz gold bar at 99.5% purity (9950 basis points): $100 \times 9950 / 10,000 = 99.5$ fine ozt
- 100 g gold bar (≈ 3.215 ozt) at 99.99% purity: $3.215 \times 9999 / 10,000 = 3.214678528$ fine ozt

Examples (Silver Bars):

- 1000 toz silver bar at 99.9% purity (9990 basis points): $1000 \times 9990 / 10,000 = 999$ fine ozt
- 100 toz silver bar at 99.99% purity (9999 basis points): $100 \times 9999 / 10,000 = 99.99$ fine ozt

USG Minting/Burning: When collateralizing, uncollateralizing, or claiming, the system mints or burns exactly the fine metal ounces as USG tokens (exemplified by fine gold ounces for gold bars), ensuring the 1:1 backing invariant. Redemption does not touch USG as only uncollateralized bars can be redeemed.

2. Mathematically Enforced 1:1 Backing Invariant

Critical Innovation: The system maintains a mathematical guarantee that:

$$\text{totalUSGSupply} == \text{sum}(\text{fineGoldOzt in all collateralized GoldBar contracts})$$

Enforcement Mechanism:

- When a bar is collateralized: USG minted = fine metal ounces of that bar (exemplified by fine gold ounces for gold bars)
- When a collateralized bar is claimed: USG burned = fine metal ounces of that bar

- Collateralized bars cannot be redeemed - they must first be uncollateralized (by owner directly or via claim) before redemption
- When a collateralized bar is forfeited: USG burned = fine metal ounces of that bar

Merkle Tree Integration: Only collateralized precious metal bar contracts (exemplified by GoldBar contracts for gold) appear in the Proof-of-Reserves Merkle tree. When a transaction changes backing state (collateralize, uncollateralize, claim, forfeiture), the vault provides a new Merkle root as part of the transaction. The Merkle root is updated atomically with every collateralization state change. Note: Consignment does not require a merkle root update because consignment creates an uncollateralized bar and there are no changes in USG token supply. Redemption does not affect PoR as only uncollateralized bars can be redeemed. The system applies equally to gold, silver, platinum, palladium, and other precious metals stored in vaults.

Mathematical Proof: Since every collateralization action mints/burns exactly the fine metal ounces (exemplified by fine gold ounces for gold bars), and only collateralized bars are counted in the Merkle tree, the invariant is maintained by construction. The Merkle root can be computed from on-chain precious metal bar contract data (exemplified by GoldBar contracts), enabling independent verification. This applies equally to gold, silver, platinum, palladium, and other precious metals stored in vaults.

3. Tiered Percentage-Based Storage Fee System

Critical Innovation: Storage fees are calculated as a percentage of gross weight (troy ounces), not fine ounces and not USD value. This eliminates the need for price oracles and ensures consistent fee calculation regardless of precious metal type or purity (exemplified by gold purity, but applies equally to silver, platinum, palladium, etc.).

Asset-Type-Specific Tiers: Each asset type (gold bullion, silver bullion, gold coins, etc.) has its own separate tiered fee structure configured by the vault.

Tier Structure Example:

- Gold Bullion: <5000 toz = 0.02%, 5000-<10000 toz = 0.018%, 10000+ toz = 0.015%
- Silver Bullion: <5000 toz = 0.015%, 5000-<10000 toz = 0.012%, 10000+ toz = 0.010%

Tier Determination: Tiers are determined by the total gross weight of all precious metal bar contracts (exemplified by GoldBar contracts for gold) of that asset type held by a user at a specific vault.

Fee Calculation: Annual storage fee = gross weight (troy ounces) × asset-type-specific tier percentage rate

Examples:

- User holds 100 toz gold bullion and is in the <5000 toz tier (0.02%) → annual fee = $100 \times 0.0002 = 0.02$ toz USG
- User holds 1000 toz silver bullion and is in the <5000 toz tier (0.015%) → annual fee = $1000 \times 0.00015 = 0.15$ toz USG

4. Per-User Prepaid Periods with Automatic Tier Recalculation

Critical Innovation: The system maintains one prepaid period per user per asset type per vault (rather than per precious metal bar contract, exemplified by GoldBar contracts). This enables efficient tier recalculation when users cross tier boundaries. This applies equally to gold, silver, platinum, palladium, and other precious metals stored in vaults.

Prepaid Period Structure:

- One **StorageFeeEscrow** contract per user per asset type per vault
- Tracks total gross weight of all bars of that asset type
- Stores locked-in rate at time of prepayment
- Calculates monthly fee based on locked-in rate

Automatic Tier Recalculation Algorithm:

When a user's total holdings cross a tier boundary (e.g., claiming a new bar moves them from 99 to 100 bars, changing tier from 0.02% to 0.018%):

1. **Calculate Prorated Refund:** Refund for time elapsed in current period at old tier rate
2. **Determine New Tier:** Based on new total gross weight
3. **Calculate New Annual Fee:** New total weight $\times$ new tier rate
4. **Calculate New Monthly Fee:** New annual fee / 12
5. **Calculate Months Covered:** (remaining balance + prorated refund) / new monthly fee
6. **If Months Covered $\geq$ 12:** Create new full-year period, refund excess
7. **If Months Covered $<$ 12:** Extend period end date based on what remaining balance covers at new rate

Key Benefit: All bars owned by a user pay the same tier rate, reflecting their total holdings. When tier changes occur mid-period and remaining balance is insufficient for a full year at the new tier, the period end date is extended rather than requiring additional payment.

Example Scenario: User has 100 bars (5000 toz) at 0.018% tier with 3 months remaining. User claims 1 bar (50 toz), now has 99 bars (4950 toz) $\rightarrow$ moves to 0.02% tier (higher rate). System calculates prorated refund, adds to remaining balance, calculates months covered at new rate, and extends period end date accordingly.

5. Risk-Based KYC Expiration and Renewal

Critical Innovation: KYC validity periods are risk-based, with automated restrictions and renewal workflows.

Validity Periods:

- Low-risk customers: 3 years validity
- Medium-risk customers: 2 years validity
- High-risk customers: 1 year validity
- Risk classification determined by KYC provider during initial review

Grace Period: 30-day grace period after expiration during which users receive warnings but can still perform actions.

Automated Restrictions: When KYC expires (after grace period), the following actions are blocked:

- Deposit requests (physical movement to vault)
- Consignment requests (on-chain custody placement)
- Claim / Title Transfer requests
- Redemption requests
- Vault-to-vault migration requests

Allowed Actions (read-only access):

- View GoldBar holdings
- Transfer USG (fungible token transfers)
- View Proof of Reserves
- Check account balances
- Receive USG from other parties

Renewal Process:

1. User re-completes KYC verification
2. Oracle receives updated `expiresAt` timestamp
3. Oracle updates `KYCProof` contract or creates new contract
4. Full access restored immediately upon successful renewal

Pre-Expiration Notifications: Users receive automated warnings at 90 days, 30 days, and 7 days before expiration, plus final warning on expiration date and grace period warnings.

6. Information Request Workflow

Critical Innovation: Structured off-chain communication with on-chain state transitions for deposit/consignment resolution.

State Transition: When vault needs additional information, vault exercises `EnterInformationRequest` choice (confirmed by Guardian approval) → request state changes to "information_request". This is the only on-chain transaction for entering this state.

Off-Chain Communication:

- Vault creates information requests in off-chain database with unique `infoRequestId`
- User submits responses via API/UI, stored off-chain
- Vault retrieves responses via API using `infoRequestId`
- Back-and-forth can repeat multiple times with no on-chain transactions

Final Resolution (on-chain):

- **Resolution:** Vault exercises **ApproveToShip** or proceeds to consignment → state changes from "information_request" to "approved" or "consigned"
- **Complete Denial:** Vault exercises **DenyRequest** or **DenyAfterReceipt** → state changes from "information_request" to "denied"

Key Benefit: Only state transitions are on-chain (providing immutable audit trail), while actual communication is stored off-chain for efficiency and to avoid on-chain storage costs.

7. Atomic Legal Forfeiture Execution

Critical Innovation: Encrypted court order storage with Guardian verification and atomic execution preserving 1:1 backing invariant.

Document Handling & Encryption:

- Court orders encrypted using hybrid encryption scheme (ECIES-AES256)
- Documents encrypted with symmetric key, which is then encrypted to each Guardian's public key individually
- Encrypted document package stored on IPFS/Arweave → permanent CID
- On-chain storage includes: **proofCID** (IPFS CID), **documentHash** (SHA-256 hash for integrity verification)

Guardian Verification Process:

1. Each Guardian retrieves encrypted document package from IPFS using **proofCID**
2. Guardian decrypts using their private key
3. Guardian verifies:
 - Document integrity: on-chain **documentHash** matches decrypted document hash
 - Court order validity: dates, signatures, jurisdiction, enforceability
 - Bar identification: serial number and/or barId matches order
 - Recipient address: matches designated recipient in court order

Atomic Execution (single transaction):

- If bar is **collateralized**:
 - Remove Merkle leaf
 - Burn exact fine-gold ounces of USG
 - Submit new vault + global PoR root atomically
- If bar is **uncollateralized**: No USG touch required
- Force-transfer GoldBar contract to recipient (government, agency, etc.)
- Archive ForcedRedemptionRequest contract with immutable record

Key Guarantee: 1:1 backing never breaks — USG supply shrinks precisely by the forfeited fine ounces.

8. Deposit/Consignment Separation

Critical Innovation: Distinct flows for physical movement (deposit) and on-chain custody placement (consignment).

Deposit + Consignment Flow (Gold Not at Vault):

1. User submits `DepositRequest` with all fees escrowed
2. Vault reviews and exercises `ApproveToShip` (Guardian approval required)
3. User ships physical bar to vault
4. Vault receives and verifies bar
5. Vault exercises `ConsignGoldBar` with documentation (Guardian approval required) → creates uncollateralized GoldBar contract
 - **Vault MUST provide documentation:** Vault uploads photos, certificate to IPFS and provides IPFS CID

Consignment Only Flow (Gold Already at Vault):

1. User submits `ConsignmentRequest` with all fees escrowed (transaction can be initiated by vault OR user)
2. Vault reviews and exercises `VerifyAsset` (Guardian approval required) - vault verifies bar is in their custody
3. Vault exercises `ConsignGoldBar` with documentation (Guardian approval required) → creates uncollateralized GoldBar contract
 - **Vault MUST provide documentation:** Vault uploads photos, certificate to IPFS and provides IPFS CID

Key Points:

- Transaction can be initiated by vault OR user
- **Vault MUST always provide documentation** (photos, certificate, IPFS CID) regardless of who initiates
- The bridge from physical to digital happens when vault submits a blockchain transaction with documentation
- Separating deposit (physical movement) from consignment (on-chain custody) allows for more efficient workflows when gold is already at the vault

9. Collateralization with Automatic Public Claim

Critical Innovation: When a bar is collateralized, public claim is automatically enabled, making the bar immediately claimable by any KYC'd party.

Collateralization Flow:

1. GoldBar owner exercises `Collateralize` choice
2. Contract state updates atomically:
 - `collateralized = True`
 - `publicClaimEnabled = True` (automatically enabled)
3. System calculates fine gold ounces: $\text{fineGoldOzt} = \text{grossWeightOzt} \times \text{purity} / 10,000$
4. System mints exact fine gold ounces as USG to owner

5. Vault provides new Merkle root as part of transaction (computed from on-chain GoldBar contracts) → bar is now collateralized
6. Proof-of-Reserves contract reflects this bar as backing circulating USG

Uncollateralization Flow (Owner-Only, exemplified by GoldBar contracts for gold bars):

1. Precious metal bar owner (exemplified as GoldBar owner) exercises **Uncollateralize** choice on their own collateralized bar (if bar hasn't been claimed)
2. System checks: Owner is the current owner and bar is collateralized
3. System calculates fine metal ounces: $\text{fineMetalOzt} = \text{grossWeightOzt} \times \text{purity} / 10,000$ (exemplified by fine gold ounces for gold bars)
4. System burns exact fine metal ounces of USG from owner (or requires owner to provide USG to burn)
5. Contract state updates atomically:
 - **collateralized = False**
 - **publicClaimEnabled** can now be controlled by owner
6. Vault provides new Merkle root as part of transaction (removes bar from tree) → bar is now uncollateralized
7. Proof-of-Reserves contract updated to reflect removal of bar from backing
8. This applies equally to gold, silver, platinum, palladium, and other precious metals stored in vaults

Important: Once collateralized, the bar is immediately claimable by any KYC'd party. The owner cannot disable public claim while the bar is collateralized. To regain control over public claim settings, the owner must first uncollateralize the bar. The owner can uncollateralize their own collateralized bar directly (if it hasn't been claimed), or the bar can be uncollateralized via claim (by the owner or someone else).

DRAWINGS

FIG. 1: System Architecture

Description:

FIG. 1 illustrates the system architecture showing the relationship between physical precious metal bars (exemplified by gold bars), precious metal bar contracts (exemplified by GoldBar contracts), USG tokens, and the Proof-of-Reserves system. While gold bars are shown as the primary example, the system applies equally to silver, platinum, palladium, and other precious metals stored in vaults.

Elements:

- **Physical Precious Metal Bars** (left): Physical precious metal bars (exemplified by gold bars) stored in vaults
- **Precious Metal Bar Contracts** (center): Distributed ledger contracts (exemplified by GoldBar contracts for gold) representing specific bars

- **USG Tokens** (right): Fungible tokens representing fine metal ounces (exemplified by fine gold ounces for gold bars)
- **Proof-of-Reserves Merkle Tree** (bottom): Merkle tree containing only collateralized bars
- **StorageFeeEscrow Contracts** (bottom-right): Per-user prepaid periods per asset type per vault
- **KYCProof Contracts** (top): Risk-based KYC credentials with expiration

Connections:

- Physical bars → Precious metal bar contracts (exemplified by GoldBar contracts) (via deposit/consignment)
- Precious metal bar contracts → USG tokens (via collateralization, 1:1 fine metal ounces, exemplified by fine gold ounces for gold bars)
- Collateralized precious metal bar contracts → Merkle tree leaves
- Users → StorageFeeEscrow contracts (prepaid periods)
- Users → KYCProof contracts (compliance)

Note: While gold bars are shown as the primary example, the same architecture applies to silver, platinum, palladium, and other precious metals stored in vaults.

Layout: Three main sections: Physical (left), Contracts (center), Tokens (right), with supporting systems below.

FIG. 2: Fine Metal Ounces Calculation Flowchart

Description:

FIG. 2 is a flowchart showing the fine metal ounces calculation process (exemplified by fine gold ounces for gold bars) for collateralization, claim, and redemption. While gold bars are shown as the primary example, the same calculation applies to silver, platinum, palladium, and other precious metals stored in vaults.

Flow Steps:

1. **START** (top, rounded rectangle)
2. **Input Bar Data** (rectangle)
 - grossWeightOzt
 - purity (basis points)
 - Arrow down to:
3. **Calculate Fine Metal Ounces** (rectangle, shows formula)
 - $\text{fineMetalOzt} = \text{grossWeightOzt} \times \text{purity} / 10,000$ (exemplified by fineGoldOzt for gold bars)
 - Note: "Same calculation applies to silver, platinum, palladium, etc."
 - Arrow down to:
4. **Check Action Type** (diamond decision)
 - **Collateralize?** → Arrow right to **Mint USG** (rectangle)
 - **Uncollateralize?** → Arrow down to **Burn USG** (rectangle)

- **Claim?** → Arrow down to **Burn USG** (rectangle)
- **Redeem?** → Arrow down to **No USG Burn** (rectangle, note: only uncollateralized bars can be redeemed)
- **Forfeit?** → Arrow down to **Burn USG** (rectangle)
- 5. **Update Merkle Tree** (rectangle, conditional)
 - **If Collateralize:** Add leaf
 - **If Uncollateralize/Claim/Forfeit (collateralized):** Remove leaf
 - **Note:** Redemption does not affect PoR as only uncollateralized bars can be redeemed
 - Arrow down to:
- 6. **Update Proof-of-Reserves** (rectangle)
 - Update vault PoR root
 - Update global PoR aggregate
 - Arrow down to:
- 7. **END** (bottom, rounded rectangle)

Key Elements:

- Formula prominently displayed: $\text{fineGoldOzt} = \text{grossWeightOzt} \times \text{purity} / 10,000$
- All actions maintain 1:1 backing invariant
- Merkle tree updates are atomic with USG mint/burn

Layout: Top-to-bottom flowchart with START at top, main flow down center, action-specific paths branching right/down.

FIG. 3: Tier Recalculation Flowchart

Description:

FIG. 3 is a detailed flowchart showing the automatic tier recalculation process when user holdings cross tier boundaries.

Flow Steps:

1. **START** (top, rounded rectangle)
2. **Title Change Event** (rectangle)
 - Deposit or Claim action
 - Arrow down to:
3. **Calculate New Total Weight** (rectangle)
 - $\text{newTotalWeight} = \text{currentTotalWeight} \pm \text{changedBarWeight}$
 - Arrow down to:
4. **Determine New Tier** (rectangle)
 - Lookup tier based on newTotalWeight and assetType
 - Arrow down to:
5. **Check Tier Changed?** (diamond decision)
 - **No:** Arrow down to **Update Weight Only** (rectangle) → **END**

- **Yes:** Arrow down to:
- 6. **Calculate Prorated Refund** (rectangle)
 - $\text{proratedRefund} = (\text{timeElapsed} / 365 \text{ days}) \times \text{oldAnnualFee}$
 - Arrow down to:
- 7. **Calculate New Annual Fee** (rectangle)
 - $\text{newAnnualFee} = \text{newTotalWeight} \times \text{newTierRate}$
 - Arrow down to:
- 8. **Calculate New Monthly Fee** (rectangle)
 - $\text{newMonthlyFee} = \text{newAnnualFee} / 12$
 - Arrow down to:
- 9. **Calculate Total Available Balance** (rectangle)
 - $\text{totalBalance} = \text{remainingBalance} + \text{proratedRefund}$
 - Arrow down to:
- 10. **Calculate Months Covered** (rectangle)
 - $\text{monthsCovered} = \text{totalBalance} / \text{newMonthlyFee}$
 - Arrow down to:
- 11. **Check Months Covered** (diamond decision)
 - **monthsCovered** $\geq$ 12? → Yes: Arrow down to **Create New Full-Year Period** (rectangle)
 - **No:** Arrow down to **Extend Period End Date** (rectangle)
- 12. **Update StorageFeeEscrow Contract** (rectangle)
 - Update totalGrossWeight, tier, lockedInRate, prepaidUntil
 - Arrow down to:
- 13. **END** (bottom, rounded rectangle)

Key Elements:

- Formula boxes showing calculations
- Decision points clearly marked
- Two paths: full-year period creation vs. period extension

Layout: Top-to-bottom flowchart with START at top, main flow down center, decision branches clearly marked.

FIG. 4: KYC Expiration and Renewal Flowchart

Description:

FIG. 4 illustrates the risk-based KYC expiration system with automated restrictions and renewal workflows.

Flow Steps:

1. **START** (top, rounded rectangle)
2. **User Action Requested** (rectangle)
 - Deposit, Claim, Redemption, Migration
 - Arrow down to:

3. **Check KYCProof Contract** (diamond decision)
 - **KYCProof exists?** → No: Arrow to **BLOCK ACTION** (terminator, left)
 - **Yes:** Arrow down to:
4. **Check Expiration Status** (diamond decision)
 - **expiresAt > currentTime?** → Yes: Arrow down to **ALLOW ACTION** (rectangle) → **END**
 - **No:** Arrow down to:
5. **Check Grace Period** (diamond decision)
 - **Within 30-day grace period?** → Yes: Arrow down to **WARN BUT ALLOW** (rectangle) → **END**
 - **No:** Arrow down to:
6. **Check Action Type** (diamond decision)
 - **Title-changing action?** → Yes: Arrow to **BLOCK ACTION** (terminator, left)
 - **Read-only action?** → Arrow down to **ALLOW ACTION** (rectangle) → **END**
7. **User Receives Renewal Notification** (rectangle)
 - Warnings at 90, 30, 7 days before expiration
 - Arrow down to:
8. **User Re-completes KYC** (rectangle)
 - Arrow down to:
9. **Oracle Updates KYCProof** (rectangle)
 - Updates expiresAt timestamp
 - Arrow down to:
10. **Full Access Restored** (rectangle)
 - Arrow down to:
11. **END** (bottom, rounded rectangle)

Key Elements:

- Risk-based validity periods shown (1-3 years)
- Grace period clearly marked
- Action type differentiation (title-changing vs. read-only)

Layout: Top-to-bottom flowchart with START at top, main flow down center, blocking paths on left, renewal path on right.

CLAIMS

System Claims

1. A distributed ledger tokenization system for physical precious metal assets stored in vaults (including but not limited to gold, silver, platinum, and palladium bars) comprising:
 - A precious metal bar contract template (exemplified by GoldBar contract template for gold bars) representing specific physical precious metal bars with gross weight and purity stored as immutable values
 - A fine metal ounces calculation algorithm (exemplified by fine gold ounces for gold bars) using the formula: $\text{fineMetalOzt} = \text{grossWeightOzt} \times \text{purity}$

/ 10,000 where purity is stored in basis points, applicable to gold, silver, platinum, palladium, and other precious metals

- A mathematically enforced 1:1 backing invariant where total USG token supply equals the exact sum of fine metal ounces (exemplified by fine gold ounces for gold bars) in all collateralized precious metal bar contracts (exemplified by GoldBar contracts)
 - A tiered percentage-based storage fee system with asset-type-specific tier structures based on total gross weight holdings per user per asset type per vault, supporting gold, silver, platinum, palladium, and other precious metals
 - An automatic tier recalculation mechanism that recalculates prepaid periods when user holdings cross tier boundaries
2. The system of claim 1, wherein the fine metal ounces calculation supports multiple purities for any precious metal, including but not limited to gold purities of 99.99%, 99.95%, and 99.5%, silver purities of 99.9% and 99.99%, and similar purity standards for platinum, palladium, and other precious metals.
 3. The system of claim 1, wherein the 1:1 backing invariant is maintained by minting or burning exactly the fine metal ounces (exemplified by fine gold ounces for gold bars) as USG tokens during collateralization, uncollateralization, claim, and forfeiture operations, applicable to gold, silver, platinum, palladium, and other precious metals stored in vaults.
 4. The system of claim 1, wherein the tiered storage fee system maintains one prepaid period per user per asset type per vault, enabling efficient tier recalculation when holdings cross tier boundaries.
 5. The system of claim 1, wherein automatic tier recalculation calculates prorated refunds, determines new tier rates, and either creates new full-year periods or extends existing period end dates based on remaining balance coverage at new tier rates.
 6. A risk-based KYC expiration and renewal system comprising:
 - KYCProof contracts storing expiration timestamps and risk levels (LOW, MEDIUM, HIGH)
 - Risk-based validity periods: 3 years for low-risk, 2 years for medium-risk, 1 year for high-risk
 - A 30-day grace period after expiration during which warnings are issued but actions remain allowed
 - Automated blocking of title-changing actions when KYC expires (after grace period)
 - Automated renewal workflow restoring full access upon KYC re-completion
 7. An information request workflow system comprising:
 - On-chain state transitions marking deposit/consignment requests as "information_request"
 - Off-chain database storage for information requests and responses linked by unique infoRequestId

- Final resolution on-chain transitioning state from "information_request" to "approved", "consigned", or "denied"
 - Immutable audit trail of state transitions while keeping communication off-chain for efficiency
8. An atomic legal forfeiture execution system comprising:
- Encrypted court order storage using hybrid encryption (ECIES-AES256) with multi-recipient key distribution
 - Guardian verification process decrypting and verifying court orders before execution
 - Atomic execution preserving 1:1 backing invariant by burning exact fine-gold ounces of USG when collateralized bars are forfeited
 - Immutable contract archive with encrypted document CID linking execution to court orders
9. A deposit/consignment separation system comprising:
- DepositRequest template for physical movement of precious metal (exemplified by gold) to vault
 - ConsignmentRequest template for on-chain custody placement when precious metal (exemplified by gold) already at vault
 - Distinct workflows allowing consignment without physical movement
 - Guardian approval required for all vault controller choices in both flows
 - Applicable to gold, silver, platinum, palladium, and other precious metals stored in vaults
10. A collateralization system with automatic public claim comprising:
- Automatic enabling of public claim when bar is collateralized
 - Immediate claimability by any KYC'd party for collateralized bars
 - Inability to disable public claim while bar remains collateralized
 - Owner can uncollateralize their own collateralized bar directly (if not claimed) or via claim to regain control over public claim settings

Method Claims

11. A method for calculating fine metal ounces supporting multiple purities for any precious metal comprising:
- Storing gross weight in troy ounces and purity in basis points in precious metal bar contracts (exemplified by GoldBar contracts for gold)
 - Calculating fine metal ounces using: $\text{fineMetalOzt} = \text{grossWeightOzt} \times \text{purity} / 10,000$ (exemplified by fine gold ounces for gold bars)
 - Minting or burning exactly the calculated fine metal ounces as USG tokens during collateralization state changes
 - Applying the same calculation method to gold, silver, platinum, palladium, and other precious metals stored in vaults

12. A method for maintaining mathematically enforced 1:1 backing comprising:

- Tracking all collateralized precious metal bar contracts (exemplified by GoldBar contracts for gold) in a Merkle tree
- Minting USG equal to fine metal ounces when bars are collateralized (exemplified by fine gold ounces for gold bars)
- Burning USG equal to fine metal ounces when bars are uncollateralized (by owner), claimed, or forfeited (collateralized bars cannot be redeemed - must be uncollateralized first)
- Updating Merkle tree and Proof-of-Reserves roots atomically with USG mint/burn operations
- Applying the same backing mechanism to gold, silver, platinum, palladium, and other precious metals stored in vaults

13. A method for automatic tier recalculation comprising:

- Detecting when user total gross weight crosses tier boundaries
- Calculating prorated refunds for time elapsed in current period
- Determining new tier rate based on new total weight
- Calculating months covered by remaining balance at new tier rate
- Creating new full-year period if months covered ≥ 12 , or extending period end date if months covered < 12

14. A method for risk-based KYC expiration management comprising:

- Assigning validity periods based on risk level (1-3 years)
- Issuing automated warnings before expiration (90, 30, 7 days)
- Implementing 30-day grace period with warnings
- Blocking title-changing actions when KYC expires (after grace period)
- Allowing read-only actions (view holdings, transfer USG) when KYC expires
- Restoring full access upon KYC renewal

15. A method for atomic legal forfeiture execution comprising:

- Encrypting court orders using hybrid encryption with multi-recipient key distribution
- Storing encrypted documents on IPFS/Arweave with permanent CID
- Storing document hash on-chain for integrity verification
- Guardians decrypting and verifying court orders independently
- Executing forfeiture atomically: removing Merkle leaf, burning USG, force-transferring GoldBar contract
- Preserving 1:1 backing invariant throughout execution

CONCLUSION

The Precious Metal Tokenization System (exemplified by Gold Bars v1.3) provides a comprehensive solution for tokenizing physical precious metals (including gold, silver, platinum, palladium, and other precious metals) stored in vaults on distributed ledgers, with key innovations including:

1. Mathematically enforced 1:1 backing with fine metal ounces calculation supporting multiple purities for any precious metal (exemplified by fine gold ounces for gold bars)
2. Tiered percentage-based storage fees with automatic tier recalculation
3. Risk-based KYC expiration and renewal with automated restrictions
4. Information request workflow with off-chain communication and on-chain state tracking
5. Atomic legal forfeiture execution preserving backing invariants
6. Deposit/consignment separation enabling efficient workflows
7. Collateralization with automatic public claim enabling immediate liquidity

These innovations address critical limitations in existing tokenization systems and provide a robust foundation for secure, compliant, and efficient gold tokenization on distributed ledgers.

PROVISIONAL PATENT APPLICATION

CONDITIONAL CLAIM PROTOCOL - DISTRIBUTED LEDGER TOKENIZATION SYSTEM FOR REAL WORLD ASSETS

Title: Distributed Ledger Tokenization System with Conditional Claims, Configurable Asset Class Support, Tiered Storage Fees, and Automated Compliance for Physical Real World Assets

Inventor(s): [To be filled]

Assignee: [To be filled]

Date: [To be filled]

Related Application: This application references the Gold Bars v1.3 Provisional Patent Application for the gold-specific implementation, and the Guardian Multi-Signature Approval System for distributed ledger governance.

ABSTRACT

A system and method for tokenizing physical Real World Assets (RWAs) stored in custodians on distributed ledger technology with mathematically enforced 1:1 backing, configurable asset class support, tiered percentage-based storage fee system with automatic tier recalculation, configurable unit value calculation supporting multiple asset classes, risk-based KYC expiration and renewal, and atomic legal forfeiture execution. The system provides per-user prepaid storage periods with asset-type-specific tiered rates, automatic tier boundary crossing detection, and prorated refund calculations. The system supports diverse asset classes including but not limited to precious metals, real estate, art, commodities, and other valuable physical assets through a configurable asset class registry. Each asset is represented by a Conditional Claim Contract that confers legally enforceable rights to obtain full ownership and control of the physical asset once specified conditions are met. The system is designed for privacy-preserving distributed ledger networks such as Canton Network using Daml, but is applicable to any distributed ledger technology including Ethereum Virtual Machine (EVM) compatible blockchains.

FIELD OF THE INVENTION

This invention relates to distributed ledger technology, asset tokenization, and Real World Asset (RWA) tokenization. More specifically, it relates to systems and methods for tokenizing physical Real World Assets stored in custodians with mathematically enforced backing ratios, configurable asset class support, automated storage fee management, and compliance controls. The invention applies to diverse asset classes including but not limited to precious metals, real estate, art, commodities, and other valuable physical assets.

BACKGROUND OF THE INVENTION

Traditional systems for tokenizing physical Real World Assets stored in custodians face several limitations:

1. **No Mathematical Backing Guarantee:** Existing systems cannot mathematically enforce that token supply equals backing assets at all times
2. **Single Asset Class Support:** Systems are typically designed for one asset class (e.g., gold, real estate) and cannot support diverse asset types
3. **Inflexible Unit Calculations:** Systems use hardcoded unit calculation formulas that cannot accommodate different asset classes with different quality metrics
4. **Inefficient Storage Fee Models:** Storage fees are typically flat-rate or per-asset, not accounting for volume discounts or asset-type differences
5. **No Automatic Tier Recalculation:** When holdings change, storage fee tiers must be manually recalculated, creating opportunities for errors
6. **Custodial IOU Structures:** Systems typically use custodial IOU structures that create issuer liability and centralization risks
7. **No KYC Expiration Management:** KYC credentials remain valid indefinitely, creating compliance risks
8. **No Atomic Legal Forfeiture:** Court orders and regulatory actions cannot be executed atomically while preserving backing invariants
9. **No Information Request Workflow:** Deposit/consignment processes lack structured off-chain communication with on-chain state tracking

The present invention addresses these limitations by providing a comprehensive tokenization system with mathematically enforced backing, configurable asset class support, automated tier recalculation, Conditional Claim legal structure, risk-based KYC expiration, and atomic legal forfeiture execution.

SUMMARY OF THE INVENTION

The Conditional Claim Protocol is a distributed ledger-based framework that provides:

1. **Mathematically Enforced 1:1 Backing:** Total fungible token supply equals exact sum of unit values in all collateralized asset contracts at all times
2. **Configurable Asset Class Support:** Dynamic asset class registry supporting diverse asset types with configurable metadata schemas, unit calculation formulas, base quantity units, quality factor formats, and divisors
3. **Configurable Unit Value Calculation:** Asset-class-specific formulas for calculating unit value: $\text{unitValue} = \text{baseQuantity} \times \text{qualityFactor} / \text{divisor}$, supporting diverse asset classes (precious metals, real estate, art, commodities, etc.)
4. **Tiered Percentage-Based Storage Fees:** Asset-type-specific tiered fee structures based on total base quantity holdings per user per asset type per custodian
5. **Automatic Tier Recalculation:** When user holdings cross tier boundaries, entire prepaid period recalculated at new tier rate atomically
6. **Per-User Prepaid Periods:** One prepaid period per user per asset type per custodian (not per asset), enabling efficient tier management
7. **Risk-Based KYC Expiration:** KYC validity periods (1-3 years) based on risk level, with automated restrictions and renewal workflows
8. **Information Request Workflow:** Off-chain communication with on-chain state transitions for deposit/consignment resolution
9. **Atomic Legal Forfeiture:** Encrypted court order storage with Guardian verification and atomic execution preserving 1:1 backing

10. **Deposit/Consignment Separation:** Distinct flows for physical movement (deposit) and on-chain custody placement (consignment)
11. **Conditional Claim Legal Structure:** Each asset represented by Conditional Claim Contract conferring legally enforceable rights to obtain full ownership and control once specified conditions are met, avoiding custodial IOU structures

DETAILED DESCRIPTION OF THE INVENTION

System Architecture

Core Data Structures

Asset Contract (AssetContract):

- **assetId:** Unique asset identifier (serial number, VIN, parcel number, etc.)
- **custodian:** Party managing the custodian
- **owner:** Current owner party
- **assetClass:** Asset class identifier (e.g., "PRECIOUS_METALS", "REAL_ESTATE", "ART", "COMMODITIES")
- **assetMetadata:** Asset-class-specific metadata (validated against registered schema)
- **baseQuantity:** Base quantity in asset-class-specific units (troy ounces for precious metals, square feet for real estate, etc.)
- **qualityFactor:** Quality factor in asset-class-specific format (purity in basis points for precious metals, condition grade as decimal for real estate, etc.)
- **collateralized:** Boolean indicating if asset backs fungible tokens
- **publicClaimEnabled:** Boolean indicating if asset is claimable by any KYC'd party
- **ipfsCID:** IPFS content identifier for photos, certificate, metadata
- **legalHash:** SHA-256 hash of custody agreement
- **merkleLeafIndex:** Optional index in Merkle tree if collateralized

AssetClassRegistry Contract:

- **protocol:** Party managing the protocol
- **assetClasses:** Map of asset class identifiers to asset class configurations
- Each asset class configuration includes:
 - **metadataSchema:** Required fields and types for asset identification
 - **unitCalculationFormula:** Formula description for calculating unit value
 - **baseQuantityUnits:** Units for measuring base quantity
 - **qualityFactorFormat:** Format for quality factor (BASIS_POINTS, DECIMAL, GRADE)
 - **divisor:** Divisor for unit value calculation
 - **supportsFractionalization:** Whether assets can be fractionalized
 - **minBaseQuantity:** Optional minimum base quantity
 - **maxBaseQuantity:** Optional maximum base quantity

StorageFeeEscrow Contract:

- **user**: Party owning the prepaid period
- **custodian**: Party managing the custodian
- **assetType**: Asset type for this prepaid period
- **totalBaseQuantity**: Total base quantity of all assets of this asset type
- **tier**: Current tier based on total base quantity
- **lockedInRate**: Rate locked in at prepayment (basis points)
- **prepaidUntil**: Date when prepaid period expires
- **monthlyFeeAmount**: Calculated monthly fee at locked-in rate

KYCProof Contract:

- **user**: Party that completed KYC
- **providerId**: KYC provider identifier (e.g., "sumsub")
- **refId**: Provider reference ID
- **level**: KYC verification level
- **countryCode**: ISO country code
- **expiresAt**: Expiration timestamp
- **riskLevel**: Risk classification (LOW, MEDIUM, HIGH)

CustodianRegistration Contract:

- **custodian**: Party managing the custodian
- **admin**: Custodian administrator
- **custodianName**: Name of custodian
- **encryptedIpfsCID**: Encrypted custodian documentation
- **legalHash**: SHA-256 hash of custodian agreement
- **status**: Custodian status (PENDING, APPROVED, ACTIVE, SUSPENDED)
- **supportedAssetClasses**: List of asset classes this custodian supports
- Guardian fields inherited from Guardian template

Core Algorithms

1. Configurable Unit Value Calculation

Critical Innovation: The system calculates unit value using asset-class-specific formulas that support diverse asset classes. Each asset class defines its own calculation formula through the AssetClassRegistry.

Generic Formula:

$\text{unitValue} = \text{baseQuantity} \times \text{qualityFactor} / \text{divisor}$

Asset-Class-Specific Examples:

Precious Metals:

$\text{unitValue} = \text{troyOunces} \times \text{purity (basis points)} / 10,000$

- Base quantity: troy ounces
- Quality factor: purity in basis points (e.g., 9999 for 99.99%)
- Divisor: 10,000
- Example: 100 toz gold bar at 99.99% purity: $100 \times 9999 / 10,000 = 99.99$ unit value

Real Estate:

$\text{unitValue} = \text{squareFeet} \times \text{conditionGrade (decimal)} / 1$

- Base quantity: square feet
- Quality factor: condition grade as decimal (0-1, e.g., 0.95 for 95%)
- Divisor: 1
- Example: 1000 sqft property at 95% condition: $1000 \times 0.95 / 1 = 950$ unit value

Art:

$\text{unitValue} = 1 \times \text{authenticityScore (decimal)} / 1$

- Base quantity: 1 (whole piece, no fractionalization)
- Quality factor: authenticity/condition score as decimal (0-1, e.g., 0.98 for 98%)
- Divisor: 1
- Example: Artwork with 98% authenticity: $1 \times 0.98 / 1 = 0.98$ unit value

Commodities:

$\text{unitValue} = \text{quantity (units)} \times \text{grade (basis points or decimal)} / \text{divisor}$

- Base quantity: quantity in units (varies by commodity type)
- Quality factor: grade in basis points or decimal (varies by commodity type)
- Divisor: 10,000 for basis points, 1 for decimal (varies by commodity type)
- Example: 100 units at 98% grade (decimal): $100 \times 0.98 / 1 = 98$ unit value

Fungible Token Minting/Burning: When collateralizing, uncollateralizing, or claiming, the system mints or burns exactly the unit value as fungible tokens (asset-class-specific tokens), ensuring the 1:1 backing invariant.

2. Mathematically Enforced 1:1 Backing Invariant

Critical Innovation: The system maintains a mathematical guarantee that:

$\text{totalFungibleTokenSupply} == \text{sum}(\text{unitValue in all collateralized AssetContracts})$

Enforcement Mechanism:

- When an asset is collateralized: Fungible tokens minted = unit value of that asset (calculated via asset-class-specific formula)
- When a collateralized asset is claimed: Fungible tokens burned = unit value of that asset

- Collateralized assets cannot be redeemed - they must first be uncollateralized (by owner directly or via claim) before redemption
- When a collateralized asset is forfeited: Fungible tokens burned = unit value of that asset

Merkle Tree Integration: Only collateralized AssetContracts appear in the Proof-of-Reserves Merkle tree. When a transaction changes backing state (collateralize, uncollateralize, claim, forfeiture), the custodian provides a new Merkle root as part of the transaction. The Merkle root is updated atomically with every collateralization state change. Note: Consignment does not require a merkle root update because consignment creates an uncollateralized asset and there are no changes in fungible token supply. Redemption does not affect PoR as only uncollateralized assets can be redeemed.

Mathematical Proof: Since every collateralization action mints/burns exactly the unit value (calculated via asset-class-specific formula), and only collateralized assets are counted in the Merkle tree, the invariant is maintained by construction. The Merkle root can be computed from on-chain AssetContract data, enabling independent verification.

3. Asset Class Registry System

Critical Innovation: The system provides a dynamic AssetClassRegistry contract that allows registration and configuration of new asset classes without code changes.

Asset Class Configuration Components:

1. **Metadata Schema:** Each asset class defines required fields for asset identification
 - Precious Metals: serial number, refiner, mint year, weight, purity, assay certificate hash
 - Real Estate: property address, parcel number, square footage, condition grade, title certificate hash
 - Art: artist name, title, year, medium, provenance certificate hash, condition grade
 - Commodities: commodity type, grade, quantity, origin certificate hash
2. **Unit Calculation Formula:** Each asset class defines how to calculate unit value
 - Formula stored as text description
 - Actual calculation uses: $\text{unitValue} = \text{baseQuantity} \times \text{qualityFactor} / \text{divisor}$
3. **Base Quantity Units:** Asset-class-specific units for measuring base quantity
 - Precious Metals: troy ounces
 - Real Estate: square feet
 - Art: 1 (whole piece)
 - Commodities: units (varies by commodity type)
4. **Quality Factor Format:** How quality factor is represented
 - Precious Metals: BASIS_POINTS (0-10000)
 - Real Estate: DECIMAL (0-1)

- Art: DECIMAL (0-1)
- Commodities: BASIS_POINTS or DECIMAL (varies)

5. **Divisor:** Denominator for unit value calculation

- Precious Metals: 10,000 (for basis points)
- Real Estate: 1 (for decimal)
- Art: 1 (for decimal)
- Commodities: varies (10,000 for basis points, 1 for decimal)

Registration Process:

1. Protocol proposes asset class registration with configuration
2. Guardian multi-signature approval required
3. Asset class added to AssetClassRegistry contract
4. Custodians can specify which asset classes they support during registration

4. Tiered Percentage-Based Storage Fee System

Critical Innovation: Storage fees are calculated as a percentage of base quantity (asset-class-specific units), not unit value and not USD value. This eliminates the need for price oracles and ensures consistent fee calculation regardless of asset class or quality factor.

Asset-Type-Specific Tiers: Each asset type has its own separate tiered fee structure configured by the custodian.

Tier Structure Examples:

- Precious Metals (Gold Bullion): <5000 toz = 0.02%, 5000-<10000 toz = 0.018%, 10000+ toz = 0.015%
- Precious Metals (Silver Bullion): <5000 toz = 0.015%, 5000-<10000 toz = 0.012%, 10000+ toz = 0.010%
- Real Estate (Commercial): <10000 sqft = 0.1%, 10000-<100000 sqft = 0.08%, 100000+ sqft = 0.06%

Tier Determination: Tiers are determined by the total base quantity of all AssetContracts of that asset type held by a user at a specific custodian.

Fee Calculation: Annual storage fee = base quantity (asset-class-specific units) × asset-type-specific tier percentage rate

Examples:

- User holds 100 toz gold bullion and is in the <5000 toz tier (0.02%) → annual fee = $100 \times 0.0002 = 0.02$ toz fungible tokens
- User holds 1000 sqft real estate and is in the <10000 sqft tier (0.1%) → annual fee = $1000 \times 0.001 = 1.0$ sqft fungible tokens

5. Per-User Prepaid Periods with Automatic Tier Recalculation

Critical Innovation: The system maintains one prepaid period per user per asset type per custodian (rather than per AssetContract). This enables efficient tier recalculation when users cross tier boundaries.

Prepaid Period Structure:

- One **StorageFeeEscrow** contract per user per asset type per custodian
- Tracks total base quantity of all assets of that asset type
- Stores locked-in rate at time of prepayment
- Calculates monthly fee based on locked-in rate

Automatic Tier Recalculation Algorithm:

When a user's total holdings cross a tier boundary (e.g., claiming a new asset moves them from Tier 1 to Tier 2):

1. **Calculate Prorated Refund:** Refund for time elapsed in current period at old tier rate
2. **Determine New Tier:** Based on new total base quantity
3. **Calculate New Annual Fee:** New total base quantity × new tier rate
4. **Calculate New Monthly Fee:** New annual fee / 12
5. **Calculate Months Covered:** (remaining balance + prorated refund) / new monthly fee
6. **If Months Covered ≥ 12:** Create new full-year period, refund excess
7. **If Months Covered < 12:** Extend period end date based on what remaining balance covers at new rate

Key Benefit: All assets owned by a user pay the same tier rate, reflecting their total holdings. When tier changes occur mid-period and remaining balance is insufficient for a full year at the new tier, the period end date is extended rather than requiring additional payment.

Example Scenario: User has 100 assets (5000 base units) at 0.018% tier with 3 months remaining. User claims 1 asset (50 base units), now has 99 assets (4950 base units) → moves to 0.02% tier (higher rate). System calculates prorated refund, adds to remaining balance, calculates months covered at new rate, and extends period end date accordingly.

6. Risk-Based KYC Expiration and Renewal

Critical Innovation: KYC validity periods are risk-based, with automated restrictions and renewal workflows.

Validity Periods:

- Low-risk customers: 3 years validity
- Medium-risk customers: 2 years validity
- High-risk customers: 1 year validity
- Risk classification determined by KYC provider during initial review

Grace Period: 30-day grace period after expiration during which users receive warnings but can still perform actions.

Automated Restrictions: When KYC expires (after grace period), the following actions are blocked:

- Deposit requests (physical movement to custodian)
- Consignment requests (on-chain custody placement)
- Claim / Title Transfer requests
- Redemption requests
- Custodian-to-custodian migration requests

Allowed Actions (read-only access):

- View AssetContract holdings
- Transfer fungible tokens (fungible token transfers)
- View Proof of Reserves
- Check account balances
- Receive fungible tokens from other parties

Renewal Process:

1. User re-completes KYC verification
2. Oracle receives updated `expiresAt` timestamp
3. Oracle updates `KYCProof` contract or creates new contract
4. Full access restored immediately upon successful renewal

Pre-Expiration Notifications: Users receive automated warnings at 90 days, 30 days, and 7 days before expiration, plus final warning on expiration date and grace period warnings.

7. Information Request Workflow

Critical Innovation: Structured off-chain communication with on-chain state transitions for deposit/consignment resolution.

State Transition: When custodian needs additional information, custodian exercises `EnterInformationRequest` choice (confirmed by Guardian approval) → request state changes to "information_request". This is the only on-chain transaction for entering this state.

Off-Chain Communication:

- Custodian creates information requests in off-chain database with unique `infoRequestId`
- User submits responses via API/UI, stored off-chain
- Custodian retrieves responses via API using `infoRequestId`
- Back-and-forth can repeat multiple times with no on-chain transactions

Final Resolution (on-chain):

- **Resolution:** Custodian exercises `ApproveToShip` or proceeds to consignment → state changes from "information_request" to "approved" or "consigned"

- **Complete Denial:** Custodian exercises **DenyRequest** or **DenyAfterReceipt** → state changes from "information_request" to "denied"

Key Benefit: Only state transitions are on-chain (providing immutable audit trail), while actual communication is stored off-chain for efficiency and to avoid on-chain storage costs.

8. Atomic Legal Forfeiture Execution

Critical Innovation: Encrypted court order storage with Guardian verification and atomic execution preserving 1:1 backing invariant.

Document Handling & Encryption:

- Court orders encrypted using hybrid encryption scheme (ECIES-AES256)
- Documents encrypted with symmetric key, which is then encrypted to each Guardian's public key individually
- Encrypted document package stored on IPFS/Arweave → permanent CID
- On-chain storage includes: **proofCID** (IPFS CID), **documentHash** (SHA-256 hash for integrity verification)

Guardian Verification Process:

1. Each Guardian retrieves encrypted document package from IPFS using **proofCID**
2. Guardian decrypts using their private key
3. Guardian verifies:
 - Document integrity: on-chain **documentHash** matches decrypted document hash
 - Court order validity: dates, signatures, jurisdiction, enforceability
 - Asset identification: asset identifier and/or assetId matches order
 - Recipient address: matches designated recipient in court order

Atomic Execution (single transaction):

- If asset is **collateralized**:
 - Remove Merkle leaf
 - Burn exact unit value of fungible tokens (calculated via asset-class-specific formula)
 - Submit new custodian + global PoR root atomically
- If asset is **uncollateralized**: No fungible token touch required
- Force-transfer AssetContract to recipient (government, agency, etc.)
- Archive ForcedRedemptionRequest contract with immutable record

Key Guarantee: 1:1 backing never breaks — Fungible token supply shrinks precisely by the forfeited unit value.

9. Deposit/Consignment Separation

Critical Innovation: Distinct flows for physical movement (deposit) and on-chain custody placement (consignment).

Deposit + Consignment Flow (Asset Not at Custodian):

1. User submits `DepositRequest` with all fees escrowed
2. Custodian reviews and exercises `ApproveToShip` (Guardian approval required)
3. User ships physical asset to custodian
4. Custodian receives and verifies asset
5. Custodian exercises `ConsignAsset` with documentation (Guardian approval required)
→ creates uncollateralized `AssetContract`
 - **Custodian MUST provide documentation:** Custodian uploads photos, certificate, asset-class-specific documentation to IPFS and provides IPFS CID

Consignment Only Flow (Asset Already at Custodian):

1. User submits `ConsignmentRequest` with all fees escrowed (transaction can be initiated by custodian OR user)
2. Custodian reviews and exercises `VerifyAsset` (Guardian approval required) - custodian verifies asset is in their custody
3. Custodian exercises `ConsignAsset` with documentation (Guardian approval required)
→ creates uncollateralized `AssetContract`
 - **Custodian MUST provide documentation:** Custodian uploads photos, certificate, asset-class-specific documentation to IPFS and provides IPFS CID

Key Points:

- Transaction can be initiated by custodian OR user
- **Custodian MUST always provide documentation** (photos, certificate, asset-class-specific documentation, IPFS CID) regardless of who initiates
- The bridge from physical to digital happens when custodian submits a blockchain transaction with documentation
- Separating deposit (physical movement) from consignment (on-chain custody) allows for more efficient workflows when assets are already at the custodian

10. Collateralization with Automatic Public Claim

Critical Innovation: When an asset is collateralized, public claim is automatically enabled, making the asset immediately claimable by any KYC'd party.

Collateralization Flow:

1. `AssetContract` owner exercises `Collateralize` choice
2. Contract state updates atomically:
 - `collateralized = True`
 - `publicClaimEnabled = True` (automatically enabled)
3. System calculates unit value via asset-class-specific formula: `unitValue = baseQuantity × qualityFactor / divisor`
4. System mints exact unit value as fungible tokens (asset-class-specific) to owner
5. Custodian provides new Merkle root as part of transaction (computed from on-chain `AssetContracts`) → asset is now collateralized

6. Proof-of-Reserves contract reflects this asset as backing circulating fungible tokens

Uncollateralization Flow (Owner-Only):

1. AssetContract owner exercises **Uncollateralize** choice on their own collateralized asset (if asset hasn't been claimed)
2. System checks: Owner is the current owner and asset is collateralized
3. System calculates unit value via asset-class-specific formula: $\text{unitValue} = \text{baseQuantity} \times \text{qualityFactor} / \text{divisor}$
4. System burns exact unit value of fungible tokens from owner (or requires owner to provide fungible tokens to burn)
5. Contract state updates atomically:
 - **collateralized = False**
 - **publicClaimEnabled** can now be controlled by owner
6. Custodian provides new Merkle root as part of transaction (removes asset from tree) → asset is now uncollateralized
7. Proof-of-Reserves contract updated to reflect removal of asset from backing

Important: Once collateralized, the asset is immediately claimable by any KYC'd party. The owner cannot disable public claim while the asset is collateralized. To regain control over public claim settings, the owner must first uncollateralize the asset. The owner can uncollateralize their own collateralized asset directly (if it hasn't been claimed), or the asset can be uncollateralized via claim (by the owner or someone else).

11. Conditional Claim Legal Structure

Critical Innovation: Each physical asset is represented by a Conditional Claim Contract (AssetContract) that confers a legally enforceable right to obtain full ownership and control of the physical asset once specified conditions are met.

Legal Distinctions from Custodial IOUs:

Custodial IOU Structure (Traditional Tokenization):

- Token holder owns a claim **on the issuer**, not the asset
- Issuer has balance-sheet liability
- Token represents issuer's promise to deliver
- If issuer fails, token holders are unsecured creditors
- Centralized control and single point of failure

Conditional Claim Structure (Our System):

- Token holder owns a claim **on the specific asset**, not the issuer
- No balance-sheet liability for issuer
- AssetContract represents legal rights wrapper, not issuer promise
- Ownership rights transfer directly between users without issuer involvement
- Decentralized control with multi-party governance

Conditions for Claiming Ownership:

- Presentation of the AssetContract (Conditional Claim NFT)
- Paying applicable redemption or transport fees
- Passing custodian-level KYC/AML requirements
- Complying with jurisdictional regulations

Key Legal Breakthroughs:

1. **No Custodial IOU:** The custodian never issues a custodial IOU. The AssetContract is a legal rights wrapper, not a balance-sheet liability.
2. **Protocol Not Custodian:** The protocol does not become a custodian. It merely standardizes metadata, attestations, and ownership registries.
3. **Direct Ownership Transfer:** Ownership rights are transferred directly between users without requiring involvement from the custodian until redemption.
4. **Legally Supported:** This structure has been reviewed and approved by legal counsel for the underlying legal architecture.

Legal Classification:

- **Not a Security:** Conditional Claims avoid securities classification due to their conditional nature, non-fractionalization (for whole-asset classes), and lack of profit expectation.
- **Not a Custodial Product:** The custodian does not hold assets "for" token holders. Token holders hold direct claim rights to specific assets.
- **New Legal Category:** This represents a new legal category of digital asset that bridges physical property rights and digital ownership.

DRAWINGS

FIG. 1: System Architecture

Description:

FIG. 1 illustrates the system architecture showing the relationship between physical Real World Assets, AssetContracts (Conditional Claim Contracts), fungible tokens, and the Proof-of-Reserves system. The system supports diverse asset classes including precious metals, real estate, art, commodities, and other valuable physical assets.

Elements:

- **Physical Real World Assets** (left): Physical assets stored in custodians (precious metals, real estate, art, commodities, etc.)
- **AssetContracts** (center): Distributed ledger contracts representing specific assets with asset-class-specific metadata
- **Fungible Tokens** (right): Asset-class-specific fungible tokens representing unit values calculated via asset-class-specific formulas
- **Proof-of-Reserves Merkle Tree** (bottom): Merkle tree containing only collateralized assets
- **StorageFeeEscrow Contracts** (bottom-right): Per-user prepaid periods per asset type per custodian

- **KYCProof Contracts** (top): Risk-based KYC credentials with expiration
- **AssetClassRegistry** (top-center): Dynamic registry of asset class configurations

Connections:

- Physical assets → AssetContracts (via deposit/consignment)
- AssetContracts → Fungible tokens (via collateralization, 1:1 unit value)
- Collateralized AssetContracts → Merkle tree leaves
- Users → StorageFeeEscrow contracts (prepaid periods)
- Users → KYCProof contracts (compliance)
- AssetClasses → AssetClassRegistry (configurations)

Layout: Three main sections: Physical (left), Contracts (center), Tokens (right), with supporting systems below and asset class registry above.

FIG. 2: Unit Value Calculation Flowchart

Description:

FIG. 2 is a flowchart showing the configurable unit value calculation process for collateralization, claim, and redemption across different asset classes.

Flow Steps:

1. **START** (top, rounded rectangle)
2. **Input Asset Data** (rectangle)
 - assetClass
 - baseQuantity (asset-class-specific units)
 - qualityFactor (asset-class-specific format)
 - Arrow down to:
3. **Lookup Asset Class Configuration** (rectangle)
 - Query AssetClassRegistry
 - Retrieve divisor and qualityFactorFormat
 - Arrow down to:
4. **Calculate Unit Value** (rectangle, shows formula)
 - $\text{unitValue} = \text{baseQuantity} \times \text{qualityFactor} / \text{divisor}$
 - Note: "Formula varies by asset class"
 - Arrow down to:
5. **Check Action Type** (diamond decision)
 - **Collateralize?** → Arrow right to **Mint Fungible Tokens** (rectangle)
 - **Uncollateralize?** → Arrow down to **Burn Fungible Tokens** (rectangle)
 - **Claim?** → Arrow down to **Burn Fungible Tokens** (rectangle)
 - **Redeem?** → Arrow down to **No Token Burn** (rectangle, note: only uncollateralized assets can be redeemed)
 - **Forfeit?** → Arrow down to **Burn Fungible Tokens** (rectangle)
6. **Update Merkle Tree** (rectangle, conditional)
 - **If Collateralize:** Add leaf

- **If Uncollateralize/Claim/Forfeit (collateralized):** Remove leaf
- **Note:** Redemption does not affect PoR as only uncollateralized assets can be redeemed
- Arrow down to:
- 7. **Update Proof-of-Reserves** (rectangle)
 - Update custodian PoR root
 - Update global PoR aggregate
 - Arrow down to:
- 8. **END** (bottom, rounded rectangle)

Key Elements:

- Asset class lookup step prominently displayed
- Formula box showing generic formula with asset-class-specific variations
- All actions maintain 1:1 backing invariant
- Merkle tree updates are atomic with fungible token mint/burn

Layout: Top-to-bottom flowchart with START at top, main flow down center, action-specific paths branching right/down.

FIG. 3: Asset Class Registration Flowchart

Description:

FIG. 3 is a flowchart showing the asset class registration process in the AssetClassRegistry.

Flow Steps:

1. **START** (top, rounded rectangle)
2. **Protocol Proposes Asset Class** (rectangle)
 - Define metadataSchema
 - Define unitCalculationFormula
 - Define baseQuantityUnits
 - Define qualityFactorFormat
 - Define divisor
 - Define supportsFractionalization
 - Arrow down to:
3. **Create AssetClassConfig** (rectangle)
 - Package all configuration components
 - Arrow down to:
4. **Submit Registration Request** (rectangle)
 - Exercise RegisterAssetClass choice
 - Arrow down to:
5. **Guardian Review** (diamond decision)
 - **Guardian approval threshold met?** → No: Arrow to **REJECT** (terminator, left)
 - **Yes:** Arrow down to:
6. **Register Asset Class** (rectangle)

- Add to AssetClassRegistry contract
- Arrow down to:
- 7. **Custodians Can Support** (rectangle)
 - Custodians specify support in CustodianRegistration
 - Arrow down to:
- 8. **Asset Contracts Can Use** (rectangle)
 - New AssetContracts can reference registered asset class
 - Arrow down to:
- 9. **END** (bottom, rounded rectangle)

Key Elements:

- Configuration components clearly shown
- Guardian approval requirement
- Custodian support specification
- Asset contract usage

Layout: Top-to-bottom flowchart with START at top, main flow down center, rejection path on left.

FIG. 4: Tier Recalculation Flowchart

Description:

FIG. 4 is a detailed flowchart showing the automatic tier recalculation process when user holdings cross tier boundaries.

Flow Steps:

1. **START** (top, rounded rectangle)
2. **Title Change Event** (rectangle)
 - Deposit or Claim action
 - Arrow down to:
3. **Calculate New Total Base Quantity** (rectangle)
 - $\text{newTotalBaseQuantity} = \text{currentTotalBaseQuantity} \pm \text{changedAssetBaseQuantity}$
 - Arrow down to:
4. **Determine New Tier** (rectangle)
 - Lookup tier based on newTotalBaseQuantity and assetType
 - Arrow down to:
5. **Check Tier Changed?** (diamond decision)
 - **No:** Arrow down to **Update Base Quantity Only** (rectangle) → **END**
 - **Yes:** Arrow down to:
6. **Calculate Prorated Refund** (rectangle)
 - $\text{proratedRefund} = (\text{timeElapsed} / 365 \text{ days}) \times \text{oldAnnualFee}$
 - Arrow down to:
7. **Calculate New Annual Fee** (rectangle)

- $\text{newAnnualFee} = \text{newTotalBaseQuantity} \times \text{newTierRate}$
- Arrow down to:
- 8. **Calculate New Monthly Fee** (rectangle)
 - $\text{newMonthlyFee} = \text{newAnnualFee} / 12$
 - Arrow down to:
- 9. **Calculate Total Available Balance** (rectangle)
 - $\text{totalBalance} = \text{remainingBalance} + \text{proratedRefund}$
 - Arrow down to:
- 10. **Calculate Months Covered** (rectangle)
 - $\text{monthsCovered} = \text{totalBalance} / \text{newMonthlyFee}$
 - Arrow down to:
- 11. **Check Months Covered** (diamond decision)
 - $\text{monthsCovered} \geq 12?$ → Yes: Arrow down to **Create New Full-Year Period** (rectangle)
 - No: Arrow down to **Extend Period End Date** (rectangle)
- 12. **Update StorageFeeEscrow Contract** (rectangle)
 - Update totalBaseQuantity, tier, lockedInRate, prepaidUntil
 - Arrow down to:
- 13. **END** (bottom, rounded rectangle)

Key Elements:

- Formula boxes showing calculations
- Decision points clearly marked
- Two paths: full-year period creation vs. period extension
- Base quantity terminology (not weight-specific)

Layout: Top-to-bottom flowchart with START at top, main flow down center, decision branches clearly marked.

FIG. 5: KYC Expiration and Renewal Flowchart

Description:

FIG. 5 illustrates the risk-based KYC expiration system with automated restrictions and renewal workflows.

Flow Steps:

1. **START** (top, rounded rectangle)
2. **User Action Requested** (rectangle)
 - Deposit, Claim, Redemption, Migration
 - Arrow down to:
3. **Check KYCProof Contract** (diamond decision)
 - **KYCProof exists?** → No: Arrow to **BLOCK ACTION** (terminator, left)
 - **Yes:** Arrow down to:
4. **Check Expiration Status** (diamond decision)

- **expiresAt > currentTime?** → Yes: Arrow down to **ALLOW ACTION** (rectangle) → **END**
- **No:** Arrow down to:
- 5. **Check Grace Period** (diamond decision)
 - **Within 30-day grace period?** → Yes: Arrow down to **WARN BUT ALLOW** (rectangle) → **END**
 - **No:** Arrow down to:
- 6. **Check Action Type** (diamond decision)
 - **Title-changing action?** → Yes: Arrow to **BLOCK ACTION** (terminator, left)
 - **Read-only action?** → Arrow down to **ALLOW ACTION** (rectangle) → **END**
- 7. **User Receives Renewal Notification** (rectangle)
 - Warnings at 90, 30, 7 days before expiration
 - Arrow down to:
- 8. **User Re-completes KYC** (rectangle)
 - Arrow down to:
- 9. **Oracle Updates KYCProof** (rectangle)
 - Updates expiresAt timestamp
 - Arrow down to:
- 10. **Full Access Restored** (rectangle)
 - Arrow down to:
- 11. **END** (bottom, rounded rectangle)

Key Elements:

- Risk-based validity periods shown (1-3 years)
- Grace period clearly marked
- Action type differentiation (title-changing vs. read-only)

Layout: Top-to-bottom flowchart with START at top, main flow down center, blocking paths on left, renewal path on right.

CLAIMS

System Claims

1. A distributed ledger tokenization system for physical Real World Assets stored in custodians comprising:
 - An AssetContract template representing specific physical assets with asset-class-specific metadata, base quantity, and quality factor stored as immutable values
 - An AssetClassRegistry contract managing configurable asset class definitions with metadata schemas, unit calculation formulas, base quantity units, quality factor formats, and divisors
 - A configurable unit value calculation algorithm using asset-class-specific formulas: $\text{unitValue} = \text{baseQuantity} \times \text{qualityFactor} / \text{divisor}$,

supporting diverse asset classes including precious metals, real estate, art, commodities, and other valuable physical assets

- A mathematically enforced 1:1 backing invariant where total fungible token supply equals the exact sum of unit values in all collateralized AssetContracts
- A tiered percentage-based storage fee system with asset-type-specific tier structures based on total base quantity holdings per user per asset type per custodian
- An automatic tier recalculation mechanism that recalculates prepaid periods when user holdings cross tier boundaries
- A Conditional Claim legal structure where each AssetContract confers legally enforceable rights to obtain full ownership and control of the physical asset once specified conditions are met, avoiding custodial IOU structures

2. The system of claim 1, wherein the unit value calculation supports diverse asset classes with asset-class-specific configurations:

- Precious metals: $\text{unitValue} = \text{troyOunces} \times \text{purity (basis points)} / 10,000$
- Real estate: $\text{unitValue} = \text{squareFeet} \times \text{conditionGrade (decimal)} / 1$
- Art: $\text{unitValue} = 1 \times \text{authenticityScore (decimal)} / 1$
- Commodities: $\text{unitValue} = \text{quantity} \times \text{grade} / \text{divisor}$ (varies by commodity type)

3. The system of claim 1, wherein the 1:1 backing invariant is maintained by minting or burning exactly the unit value (calculated via asset-class-specific formula) as fungible tokens during collateralization, uncollateralization, claim, and forfeiture operations.

4. The system of claim 1, wherein the tiered storage fee system maintains one prepaid period per user per asset type per custodian, enabling efficient tier recalculation when holdings cross tier boundaries.

5. The system of claim 1, wherein automatic tier recalculation calculates prorated refunds, determines new tier rates, and either creates new full-year periods or extends existing period end dates based on remaining balance coverage at new tier rates.

6. The system of claim 1, wherein the AssetClassRegistry allows dynamic registration of new asset classes without code changes, with each asset class defining its own metadata schema, unit calculation formula, base quantity units, quality factor format, and divisor.

7. A risk-based KYC expiration and renewal system comprising:

- KYCProof contracts storing expiration timestamps and risk levels (LOW, MEDIUM, HIGH)
- Risk-based validity periods: 3 years for low-risk, 2 years for medium-risk, 1 year for high-risk

- A 30-day grace period after expiration during which warnings are issued but actions remain allowed
- Automated blocking of title-changing actions when KYC expires (after grace period)
- Automated renewal workflow restoring full access upon KYC re-completion

8. An information request workflow system comprising:

- On-chain state transitions marking deposit/consignment requests as "information_request"
- Off-chain database storage for information requests and responses linked by unique infoRequestId
- Final resolution on-chain transitioning state from "information_request" to "approved", "consigned", or "denied"
- Immutable audit trail of state transitions while keeping communication off-chain for efficiency

9. An atomic legal forfeiture execution system comprising:

- Encrypted court order storage using hybrid encryption (ECIES-AES256) with multi-recipient key distribution
- Guardian verification process decrypting and verifying court orders before execution
- Atomic execution preserving 1:1 backing invariant by burning exact unit value of fungible tokens (calculated via asset-class-specific formula) when collateralized assets are forfeited
- Immutable contract archive with encrypted document CID linking execution to court orders

10. A deposit/consignment separation system comprising:

- DepositRequest template for physical movement of assets to custodian
- ConsignmentRequest template for on-chain custody placement when assets already at custodian
- Distinct workflows allowing consignment without physical movement
- Guardian approval required for all custodian controller choices in both flows

11. A collateralization system with automatic public claim comprising:

- Automatic enabling of public claim when asset is collateralized
- Immediate claimability by any KYC'd party for collateralized assets
- Inability to disable public claim while asset remains collateralized
- Owner can uncollateralize their own collateralized asset directly (if not claimed) or via claim to regain control over public claim settings

12. A Conditional Claim legal structure system comprising:

- AssetContracts that confer legally enforceable rights to obtain full ownership and control of physical assets once specified conditions are met

- Avoidance of custodial IOU structures (no balance-sheet liability for issuer)
- Direct ownership transfer between users without custodian involvement until redemption
- Legal classification avoiding securities classification due to conditional nature, non-fractionalization (for whole-asset classes), and lack of profit expectation

Method Claims

13. A method for calculating unit value supporting multiple asset classes comprising:

- Registering asset class configurations in AssetClassRegistry with metadata schemas, unit calculation formulas, base quantity units, quality factor formats, and divisors
- Storing base quantity and quality factor in AssetContracts according to asset-class-specific formats
- Calculating unit value using asset-class-specific formula: $\text{unitValue} = \text{baseQuantity} \times \text{qualityFactor} / \text{divisor}$
- Minting or burning exactly the calculated unit value as fungible tokens during collateralization state changes

14. A method for maintaining mathematically enforced 1:1 backing comprising:

- Tracking all collateralized AssetContracts in a Merkle tree
- Minting fungible tokens equal to unit value (calculated via asset-class-specific formula) when assets are collateralized
- Burning fungible tokens equal to unit value when assets are uncollateralized (by owner), claimed, or forfeited (collateralized assets cannot be redeemed - must be uncollateralized first)
- Updating Merkle tree and Proof-of-Reserves roots atomically with fungible token mint/burn operations

15. A method for automatic tier recalculation comprising:

- Detecting when user total base quantity crosses tier boundaries
- Calculating prorated refunds for time elapsed in current period
- Determining new tier rate based on new total base quantity
- Calculating months covered by remaining balance at new tier rate
- Creating new full-year period if months covered ≥ 12 , or extending period end date if months covered < 12

16. A method for risk-based KYC expiration management comprising:

- Assigning validity periods based on risk level (1-3 years)
- Issuing automated warnings before expiration (90, 30, 7 days)
- Implementing 30-day grace period with warnings
- Blocking title-changing actions when KYC expires (after grace period)
- Allowing read-only actions (view holdings, transfer fungible tokens) when KYC expires
- Restoring full access upon KYC renewal

17. A method for atomic legal forfeiture execution comprising:

- Encrypting court orders using hybrid encryption with multi-recipient key distribution
- Storing encrypted documents on IPFS/Arweave with permanent CID
- Storing document hash on-chain for integrity verification
- Guardians decrypting and verifying court orders independently
- Executing forfeiture atomically: removing Merkle leaf, burning fungible tokens (exact unit value calculated via asset-class-specific formula), force-transferring AssetContract
- Preserving 1:1 backing invariant throughout execution

18. A method for registering new asset classes comprising:

- Defining metadata schema with required fields and types
- Defining unit calculation formula description
- Defining base quantity units
- Defining quality factor format (BASIS_POINTS, DECIMAL, or GRADE)
- Defining divisor for unit value calculation
- Defining fractionalization support
- Submitting registration request with Guardian approval
- Adding asset class to AssetClassRegistry contract upon approval

CONCLUSION

The Conditional Claim Protocol provides a comprehensive solution for tokenizing physical Real World Assets stored in custodians on distributed ledgers, with key innovations including:

1. Mathematically enforced 1:1 backing with configurable unit value calculation supporting diverse asset classes
2. Dynamic asset class registry allowing registration of new asset classes without code changes
3. Tiered percentage-based storage fees with automatic tier recalculation
4. Risk-based KYC expiration and renewal with automated restrictions
5. Information request workflow with off-chain communication and on-chain state tracking
6. Atomic legal forfeiture execution preserving backing invariants
7. Deposit/consignment separation enabling efficient workflows
8. Collateralization with automatic public claim enabling immediate liquidity
9. Conditional Claim legal structure avoiding custodial IOU structures and providing direct ownership rights

These innovations address critical limitations in existing tokenization systems and provide a robust foundation for secure, compliant, and efficient RWA tokenization on distributed ledgers across diverse asset classes.

CLAIMS

What is claimed is:

1. A distributed-ledger platform for tokenizing multiple classes of real-world assets under unified multi-party governance, comprising: (a) a reusable governance library comprising (i) a sequential approval queue storing approval requests with queue positions and timeout intervals, (ii) voting logic that records approvals by authorized guardians, (iii) sequential execution logic that, absent an exception condition, permits execution of a queued request only when the request is at a front of the queue, and (iv) timeout and cancellation logic for unblocking the queue; (b) an asset class registry storing, for each asset class, base quantity units, a quality factor format, a divisor, and a linear unit-value formula; (c) asset contracts referencing the asset class registry, each asset contract storing an asset class identifier, a base quantity, and a quality factor; (d) a fungible token contract representing digital units corresponding to unit values computed from the asset contracts; (e) a proof-of-reserves contract maintaining a cryptographic commitment over collateralized asset contracts; (f) atomic operation logic configured such that collateralization and claim operations each execute state changes, token minting or burning, and cryptographic commitment updates in a single atomic ledger transaction; and (g) integration logic incorporating the governance library into at least one of the asset contracts, the proof-of-reserves contract, or a forfeiture workflow, such that designated state-changing actions require satisfaction of the sequential approval queue prior to execution.
2. The platform of claim 1, wherein the governance library governs at least one of: collateralize, uncollateralize, claim, forced redemption, forfeiture, registry update, proof-of-reserves root update, emergency pause, or migration.
3. The platform of claim 1, wherein the asset class registry defines a precious metals class having base quantity units in troy ounces, a quality factor format in basis points, and a divisor of 10,000, and wherein a precious-metal bar contract implements an asset contract instance of the precious metals class.
4. The platform of claim 1, wherein the governance library provides action-specific guardian sets and action-specific approval thresholds, enabling different governance policies for different asset classes and different operation types.
5. The platform of claim 1, further comprising a reusable compliance proof contract storing a non-personally-identifying eligibility state, a risk level, and an expiration time, and access-control logic that blocks a first set of actions and allows a second set of actions based on a validity window including a warning period and a grace period, wherein the compliance proof contract gates operations across all asset classes.
6. The platform of claim 1, further comprising legal forfeiture logic requiring multi-party guardian authorization and executing forced redemption comprising Merkle leaf removal, token burning, and force-transfer of the asset contract in a single atomic ledger transaction,

wherein the forfeiture logic operates across asset classes registered in the asset class registry.

7. The platform of claim 1, wherein each asset contract is a conditional claim instrument representing a right to obtain ownership or control of a corresponding real-world asset upon satisfaction of one or more conditions including compliance verification.

8. The platform of claim 1, further comprising disapproval voting logic that cancels an approval request when a disapproval count meets a disapproval threshold, wherein the disapproval threshold satisfies a deadlock-prevention constraint such that an approval threshold plus the disapproval threshold is greater than an approver count, making simultaneous satisfaction of both thresholds mathematically impossible.

9. The platform of claim 1, further comprising a prepaid tiered fee escrow contract configured to charge storage or custody fees for collateralized assets, the fees being computed as a function of an asset class, a base quantity, and a tier schedule, and to auto-recalculate a tier responsive to changes in asset quantity or ownership.

10. The platform of claim 1, wherein the proof-of-reserves commitment comprises a Merkle root over salted leaves such that a cryptographic salt or nonce reduces probing of whether a particular owner-asset combination exists in a Merkle tree.

11. The platform of claim 1, wherein the governance library permits selected owner-only actions to execute atomically without entering a governance queue while retaining governance gating for administrative, custody, or emergency actions.

12. The platform of claim 1, wherein the proof-of-reserves contract maintains per-custodian Merkle roots and a global aggregate root.

13. The platform of claim 1, wherein the governance library provides threshold-constraint logic requiring each approval threshold to be less than or equal to the guardian count minus one, and automatic adjustment logic that adjusts thresholds responsive to guardian addition or removal.

14. The platform of claim 1, wherein when an asset contract becomes collateralized the platform automatically enables a public-claim mode in which any compliance-verified holder of sufficient fungible tokens may claim the asset contract, and the owner cannot disable public claimability while the asset contract remains collateralized.

15. A distributed-ledger system comprising: (a) an asset class registry defining unit semantics and a linear unit-value formula for each of a plurality of asset classes; (b) asset contracts for assets in the plurality of asset classes; (c) a governance library providing a sequential approval queue with timeout expiration and cancellation; (d) collateralization logic configured to, upon satisfaction of the sequential approval queue for a collateralization action type, execute an atomic transaction comprising changing an asset contract state to collateralized, minting fungible tokens based on a unit value derived from the asset class registry, and adding a Merkle leaf to a proof-of-reserves commitment; and (e) claim logic configured to verify compliance proof validity and, upon satisfaction of applicable conditions, execute an atomic transaction comprising burning fungible tokens, removing a Merkle leaf, and transferring the asset contract to a claimant.

16. The system of claim 15, wherein the governance library governs forfeiture actions across the plurality of asset classes, requiring multi-party authorization referencing a legal order document hash before executing forced redemption.

17. The system of claim 15, wherein the governance library provides separate guardian sets for precious metals operations and real estate operations.

18. The system of claim 15, further comprising public-claim enablement logic that, when an asset contract is collateralized, forces claimability by compliance-verified parties holding sufficient fungible tokens.

19. A method of operating a distributed-ledger platform for tokenizing multiple classes of real-world assets, comprising: (a) registering an asset class in an asset class registry, the asset class defining base quantity units, a quality factor format, a divisor, and a linear unit-value formula; (b) creating an asset contract referencing the asset class and storing a base quantity and quality factor; (c) submitting a collateralization request to a sequential governance approval queue; (d) collecting approvals and, optionally, disapprovals under submission-time thresholds; (e) upon the collateralization request reaching the front of the queue and meeting an approval threshold, executing an atomic transaction comprising changing the asset contract to a collateralized state, minting fungible tokens equal to a computed unit value, and adding a Merkle leaf to a proof-of-reserves commitment; (f) receiving a claim request, verifying compliance proof validity, and executing an atomic transaction comprising burning fungible tokens, removing the Merkle leaf, and transferring the asset contract to a claimant; and (g) maintaining a backing invariant such that total fungible token supply equals a sum of unit values for collateralized asset contracts.

20. The method of claim 19, further comprising receiving a forfeiture request referencing a legal order document, submitting the forfeiture request to the governance approval queue, and upon multi-party authorization, executing forced redemption in a single atomic transaction.

21. The method of claim 19, further comprising dynamically registering a new asset class through a guardian-approved governance action without requiring code changes to the protocol.

22. A non-transitory computer-readable medium storing instructions that, when executed by one or more processors operating a distributed-ledger node, cause performance of the method of claim 19.

23. A system for programmable multi-party governance of real-world asset tokenization on a distributed ledger, comprising: (a) a governance control plane declaring protected actions and enforcing configurable approval policies prior to execution; (b) an asset registration layer capable of defining heterogeneous asset classes with per-class valuation rules; (c) a tokenization layer minting and burning fungible digital units corresponding to collateralized real-world assets; (d) a cryptographic backing layer maintaining a verifiable commitment that circulating digital units correspond to collateralized assets; and (e) a compliance layer gating asset operations on verifiable eligibility proofs, wherein the governance control plane governs at least one state-changing action across two or more of the asset registration layer, the

tokenization layer, the cryptographic backing layer, and the compliance layer.

24. The system of claim 23, wherein the governance control plane provides a self-healing sequential queue that automatically unblocks stalled requests via timeout-based expiration or approval-gated cancellation.

25. The system of claim 23, wherein the asset registration layer supports dynamic addition of new asset classes through governance-approved proposals without code changes.

26. The system of claim 23, wherein the tokenization layer enforces that each collateralize and claim operation executes atomically with cryptographic backing updates.

27. The system of claim 23, further comprising a legal forfeiture pathway requiring multi-party governance authorization, encrypted order artifacts, and invariant-preserving forced redemption.

28. The system of claim 23, wherein the compliance layer stores only non-personally-identifying eligibility data and gates operations through warning, grace, and expired phases.

29.) A distributed-ledger network for self-governing tokenized real-world assets, comprising: (a) means for representing heterogeneous real-world assets as on-chain objects with per-class valuation rules; (b) means for issuing and redeeming fungible digital units backed by the on-chain objects; (c) means for cryptographically proving that circulating digital units correspond to collateralized on-chain objects; (d) means for enforcing multi-party approval prior to execution of designated state-changing operations; and (e) means for legally forfeiting an on-chain object under multi-party authorization while preserving a backing invariant between circulating digital units and collateralized on-chain objects.

30. The network of claim 29, further comprising means for gating asset operations on verifiable, privacy-preserving compliance eligibility proofs that expire and renew independently of asset state.

31. The network of claim 29, further comprising means for dynamically registering new asset classes under governance approval without code modification.

32. A distributed-ledger system for tokenizing physical gold bars under multi-party governance, comprising: (a) a governance library comprising a sequential approval queue, voting logic, timeout logic, and cancellation logic; (b) a gold bar contract storing at least a bar identifier, a vault identifier, a gross weight, and a purity factor, the gold bar contract having a lifecycle state machine comprising at least uncollateralized and collateralized states; (c) a fungible token contract representing a quantity of fine gold units; (d) a proof-of-reserves contract maintaining a Merkle root over collateralized gold bars; (e) atomic collateralize and claim operations each executing state changes, token operations, and Merkle root updates in a single ledger transaction; and (f) governance integration logic designating at least one of consignment approval, forced redemption, emergency pause, or proof-of-reserves update as guardian-governed actions requiring satisfaction of the sequential approval queue.

33. The system of claim 32, wherein the governance library provides action-specific approval thresholds such that forced redemption requires a higher threshold than consignment approval.

34. The system of claim 32, further comprising disapproval voting logic enabling guardians to vote against a pending forfeiture request, wherein the forfeiture request is canceled when disapprovals meet a disapproval threshold satisfying a deadlock-prevention constraint.

35. The system of claim 32, further comprising emergency-action logic permitting out-of-order execution of an emergency pause action when an emergency approval threshold is met.

36. The system of claim 32, further comprising legal forfeiture logic requiring multi-party guardian authorization based on a reference to a legal order document and a document hash, executing forced redemption comprising Merkle leaf removal, token burning, and force-transfer in a single atomic transaction.

37. The system of claim 36, further comprising a hybrid encryption scheme in which the legal order document is encrypted with a symmetric key and the symmetric key is encrypted to a plurality of guardians' public keys.

38. The system of claim 32, wherein collateralize and uncollateralize are owner-only operations that execute atomically without entering the governance approval queue, while consignment approval and forced redemption are guardian-governed operations.

39. The system of claim 32, further comprising audit-trail logic recording, for each guardian-governed action, which guardians approved, time of approval, and terminal state.

40. A distributed-ledger system comprising: (a) a gold bar contract representing a physical gold bar and having collateralized and uncollateralized states; (b) a fungible token contract and a proof-of-reserves contract maintaining a Merkle root; (c) a governance approval queue configured to receive forfeiture requests, each forfeiture request including a reference to a legal order document hash; (d) voting logic enabling a plurality of guardians to approve or disapprove the forfeiture request; (e) upon the forfeiture request meeting an approval threshold and reaching the front of the queue, forfeiture execution logic that performs, in a single atomic transaction: removal of the Merkle leaf, burning of fungible tokens corresponding to the gold bar contract, and force-transfer of the gold bar contract to a designated recipient; and (f) queue advancement logic that, if the forfeiture request times out without meeting the threshold, marks it expired and advances the queue.

41. The system of claim 40, wherein burned tokens are sourced in priority order from at least one of: an owner balance, an insurance or reserve pool, or a protocol treasury reserve.

42. The system of claim 40, wherein the governance approval queue uses submission-time thresholds such that guardian additions or removals after submission do not alter the threshold applied to pending forfeiture requests.

43. A distributed-ledger protocol comprising: (a) an asset class registry storing, for each asset class, base quantity units, a quality factor format, a divisor, and a linear unit-value formula; (b)

a governance library comprising a sequential approval queue with voting logic, timeout logic, and cancellation logic, the governance library providing action-specific guardian sets and action-specific approval thresholds; (c) asset contracts referencing the asset class registry, each asset contract representing a conditional claim instrument conferring a right to obtain ownership of a corresponding real-world asset upon satisfaction of one or more conditions; (d) registration governance logic requiring satisfaction of the governance approval queue to register new asset classes in the asset class registry; and (e) lifecycle governance logic requiring satisfaction of the governance approval queue for at least one of consignment approval, forced redemption, or emergency pause of asset contracts across asset classes.

44. The protocol of claim 43, further comprising a proof-of-reserves contract maintaining per-custodian Merkle roots and a global aggregate root, the proof-of-reserves contract shared across asset classes registered in the asset class registry.

45. The protocol of claim 43, further comprising a reusable compliance proof contract storing a risk level and an expiration time, the compliance proof contract shared across asset classes and gating at least one of claim or title transfer.

46. The protocol of claim 43, wherein the governance library further comprises disapproval voting logic and emergency out-of-order execution logic for designated emergency action types.

47. The protocol of claim 43, further comprising a prepaid tiered storage fee escrow contract shared across asset classes, computing percentage-based fees as a function of base quantity per asset class definition.

48. The protocol of claim 43, wherein the governance library governs fractionalization approval such that creation of sub-claim contracts associated with a parent asset contract requires multi-party guardian authorization.

49. A distributed-ledger protocol comprising: (a) an asset class registry storing per-class unit semantics and a linear unit-value formula; (b) a sequential governance approval queue with configurable action-specific thresholds; (c) registration governance logic that: (i) receives a proposal to register a new asset class including proposed base quantity units, quality factor format, divisor, and linear unit-value formula; (ii) submits the proposal to the governance approval queue; (iii) upon multi-party approval, adds the new asset class to the registry without requiring code changes to the protocol; and (d) post-registration logic enabling creation of asset contracts referencing the newly registered asset class and minting fungible tokens based on the registered linear unit-value formula.

50. The protocol of claim 49, wherein the governance approval queue uses submission-time thresholds and timeout expiration to prevent stalled registration proposals from blocking subsequent proposals.

51. The protocol of claim 49, further comprising asset class deactivation governance logic requiring multi-party approval to disable an asset class, wherein deactivation prevents new collateralization but permits claim and redemption of existing asset contracts in the deactivated

class.

52. A distributed-ledger protocol for tokenizing physical gold bars as a registered asset class within a multi-asset tokenization framework, comprising: (a) an asset class registry storing, for a precious metals asset class, base quantity units of troy ounces, a quality factor format of basis points, and a divisor of 10,000; (b) a gold bar contract referencing the precious metals asset class and storing a bar identifier, a vault identifier, a base quantity corresponding to gross weight, and a quality factor corresponding to purity; (c) unit-value computation logic computing a fine gold ounce value as a product of gross weight and purity divided by the divisor using the linear unit-value formula from the asset class registry; (d) a fungible token contract representing fine gold ounce units; (e) a proof-of-reserves contract maintaining a Merkle root, each Merkle leaf computed as a cryptographic hash over canonical-ordered fields including at least an owner identifier, an asset identifier, a custodian identifier, a quantity value, and a nonce; (f) atomic collateralize and claim operations each executing in a single ledger transaction; and (g) public-claim enablement logic comprising a forced-on mode when collateralized and an owner-controlled mode when uncollateralized.

53. The protocol of claim 52, wherein the asset class registry further stores at least one additional non-precious-metals asset class, and wherein the fungible token contract, proof-of-reserves contract, and compliance proof contract are shared across asset classes.

54. The protocol of claim 52, further comprising compliance-gated operations including a warning period, a grace period, and an expired state in which a first set of actions are blocked and a second set of actions are allowed, the compliance gating shared with other asset classes in the registry.

55. The protocol of claim 52, wherein the gold bar contract is a conditional claim instrument conferring a right to obtain ownership of a specific physical gold bar upon satisfaction of conditions including compliance verification and sufficient fungible token balance.

56. The protocol of claim 52, further comprising legal forfeiture logic operable across asset classes, requiring multi-party authorization and executing forced redemption preserving a backing invariant.

57. The protocol of claim 52, further comprising a prepaid tiered storage fee escrow contract in which fees are computed as a percentage of base quantity using the troy ounce unit semantics from the asset class registry.

58. A distributed-ledger system comprising: (a) an asset class registry defining at least a precious metals class and a second asset class; (b) asset contracts for both classes, each storing an asset class identifier and a base quantity; (c) unit-value computation logic applying per-class linear unit-value formulas from the registry; (d) a proof-of-reserves contract maintaining: (i) per-custodian Merkle roots over collateralized asset contracts from both classes, each Merkle leaf including a cryptographically random nonce; and (ii) a global aggregate root; (e) a supply-invariant enforcement mechanism verifying that total fungible token supply across both classes equals a sum of unit values represented by Merkle leaves; and (f) atomic operations that update the per-custodian Merkle root and the global aggregate root as part of each collateralize,

claim, or forfeiture transaction.

59. The system of claim 58, wherein the per-custodian Merkle roots enable independent verification of reserves held by each custodian without revealing cross-custodian holdings.

60. The system of claim 58, wherein forfeiture of an asset contract in either class preserves the supply invariant by burning fungible tokens equal to the unit value of the forfeited asset contract.

61. A governance control plane for programmable transactions on a distributed ledger, comprising logic that: (i) declares protected actions for one or more smart contracts, (ii) assigns approval policies to the protected actions, and (iii) enforces the approval policies prior to execution of the protected actions, wherein the approval policies are configurable per action type and per contract instance.

62. The governance control plane of claim 61, wherein enforcement comprises a sequential approval queue that is self-healing by automatically unblocking stalled requests via at least one of timeout-based expiration or an approval-gated cancellation mechanism.

63. The governance control plane of claim 61, further comprising policy-based visibility rules that restrict request data to a subset of relevant parties while enabling approvals to be collected from a set of approvers.

64. A reusable governance library configured to be inherited or imported by an application smart contract, the library comprising state and methods enabling the application smart contract to mark one or more actions as requiring governance approval and to execute such actions only after satisfaction of a configurable approval policy.

65. The system of claim 61, wherein approval policies include per-action thresholds and per-action approver sets.

66. The system of claim 61, wherein approval policies auto-adjust when approvers are added or removed, while pending requests use submission-time approval parameters.

67. The system of claim 61, wherein execution produces an immutable audit trail per request.

68. A smart-contract governance library for reuse across application contracts on a distributed ledger, comprising: (a) guardian state storing (i) a set of guardians, (ii) a default approval threshold, (iii) action-specific approval thresholds, and (iv) action-specific guardian sets; (b) a request queue storing approval requests, each approval request including an action type, request data, a queue position, a submission-time threshold record, and a timeout start time; (c) voting logic that records approvals for an approval request by guardians authorized for the action type; (d) sequential execution logic that, absent an exception condition, permits execution of a queued approval request only when the queued approval request is at a front of the request queue; (e) timeout logic that, upon expiration of a configurable timeout interval without meeting the submission-time threshold record, marks the queued approval request as expired and advances the request queue to unblock processing of a subsequent request; and (f)

cancellation logic that enables removal of a blocking queued approval request responsive to a cancel request.

69. The library of claim 68, wherein the configurable timeout interval has a default time window of seven days.

70. The library of claim 68, wherein the cancellation logic permits execution of the cancel request out of order relative to the request queue, and wherein said out-of-order execution constitutes the exception condition of element (d).

71. The library of claim 70, wherein execution of the cancel request out of order requires satisfaction of an approval threshold for a cancel action type.

72. The library of claim 68, further comprising bounded cleanup logic configured to mark, archive, or delete no more than a bounded number of expired requests per ledger transaction.

73. The library of claim 68, wherein the action-specific guardian sets provide different approvers for different action types.

74. The library of claim 68, wherein the action-specific approval thresholds provide different approval counts for different action types.

75. The library of claim 68, wherein the library is incorporated into an application contract by inheritance, import, or template composition, and wherein at least one protected action of the application contract is designated as requiring guardian approval.

76. The library of claim 68, further comprising audit-trail logic configured to record, for each approval request, which guardian approved, a time of approval, and an executed, expired, or canceled terminal state.

77. A smart-contract governance system comprising: (a) a guardian set having a guardian count; (b) a default approval threshold and at least one action-specific approval threshold; (c) threshold-constraint logic requiring each approval threshold to be less than or equal to the guardian count minus one; (d) automatic adjustment logic that, responsive to a guardian addition or guardian removal, checks each approval threshold against the threshold-constraint logic and decrements any violating threshold to satisfy the threshold-constraint logic; and (e) a submission-time threshold rule in which pending requests use a threshold recorded at submission time.

78. The system of claim 77, wherein the submission-time threshold rule applies such that guardian removal affects only newly submitted requests and does not alter thresholds applied to already-pending requests.

79. The system of claim 77, wherein the automatic adjustment logic is performed atomically with the guardian addition or guardian removal in a same ledger transaction.

80. The system of claim 77, wherein the default approval threshold applies to action types lacking an action-specific approval threshold.

81. A method of operating a sequential multi-party approval queue in a smart-contract governance library, comprising: (a) receiving a request for a protected action, assigning a queue position, recording a submission-time threshold, and starting a timeout; (b) transitioning the request into an awaiting-approval state while accumulating guardian approvals; (c) upon approvals meeting the submission-time threshold, executing the protected action and advancing the queue; (d) upon expiration of the timeout without approvals meeting the submission-time threshold, marking the request expired and advancing the queue; and (e) upon approval of a cancel request, canceling the request and advancing the queue.

82. The method of claim 81, wherein the cancel request is executable out of order relative to the queue position of the canceled request.

83. The method of claim 81, further comprising recording an immutable execution record for executed, expired, and canceled requests.

84. A non-transitory computer-readable medium storing instructions that, when executed by one or more processors operating a distributed-ledger node, cause performance of the method of claim 81.

85. The library of claim 68, wherein a first class of actions are designated as guardian-governed actions and a second class of actions are designated as owner-only actions that execute atomically without entering the request queue.

86. The library of claim 85, wherein the owner-only actions include at least one of collateralize, uncollateralize, claim, or token transfer.

87. The library of claim 85, wherein the guardian-governed actions include at least one of consignment approval, forced redemption, emergency pause, or proof-of-reserves update.

88. The library of claim 68, further comprising privacy-based visibility rules such that only relevant parties to a request receive access to request data and at least one observer party receives a restricted view or report.

89. The library of claim 68, wherein the timeout logic unblocks the queue by incrementing a next-executable request identifier.

90. The library of claim 68, wherein the cancellation logic removes the canceled request from the queue and increments a next-executable request identifier.

91. The library of claim 68, wherein guardians are configurable per application contract instance to enable cross-institutional approvals without a single point of control.

92. The library of claim 68, wherein the protected action is executed as an atomic multi-party ledger operation.

93. The library of claim 68, wherein the voting logic further records disapproval votes by authorized guardians for the approval request.

94. The library of claim 93, further comprising disapproval-threshold logic that cancels the approval request upon disapprovals meeting a disapproval threshold.

95. The library of claim 94, wherein the disapproval threshold is computed as: $\text{disapprovalThreshold} = \min(\text{approvalThreshold}, \text{guardianCount} - \text{approvalThreshold} + 1)$, and wherein the disapproval threshold satisfies a deadlock-prevention constraint such that $\text{approvalThreshold} + \text{disapprovalThreshold}$ is greater than a guardian count, making simultaneous satisfaction of both thresholds mathematically impossible.

96. The library of claim 93, wherein disapproval-based cancellation executes out of order relative to the request queue.

97. The library of claim 68, further comprising emergency-action logic that permits out-of-order execution of designated emergency action types when the approval threshold for the emergency action type is met, wherein said out-of-order execution constitutes the exception condition of element (d).

98. A system for issuing circulating digital units backed by identifiable real-world assets, comprising: (i) representing each backing asset by a ledger object, (ii) issuing fungible units tied to a backing quantity derived from the ledger object, and (iii) maintaining a cryptographic commitment to a set of backing assets such that circulating supply is verifiable against the cryptographic commitment.

99. The system of claim 98, wherein when a backing asset is placed into a collateralized state that supports circulating fungible units, the system forces claimability of the backing asset by eligible unit holders to enable liquidity and price discovery, and prevents an owner from disabling such claimability while collateralized.

100. The system of claim 98, further comprising a compliance proof object storing a non-personally-identifying eligibility state and an expiration time, and gating at least one of asset claim, title transfer, or redemption based on the eligibility state.

101. The system of claim 98, wherein a state change affecting backing occurs atomically with at least one of minting or burning fungible units and updating the cryptographic commitment.

102. The system of claim 98, wherein the cryptographic commitment comprises a Merkle root over leaf hashes that include a privacy nonce.

103. The system of claim 98, wherein a legal forfeiture process uses encrypted order artifacts and multi-party authorization and preserves a backing invariant via prioritized burn sources.

104. A distributed-ledger system for tokenizing physical gold bars, comprising: (a) a gold bar contract storing at least a bar identifier, a vault identifier, a gross weight, and a purity factor; (b) a fungible token contract representing a quantity of fine gold units; (c) a proof-of-reserves contract maintaining a Merkle root representing collateralized gold bars; (d) a lifecycle state machine for the gold bar contract comprising at least uncollateralized and collateralized states; and (e) atomic operation logic configured such that: (i) a collateralize operation performs, in a

single atomic ledger transaction, a state change to collateralized, minting of a quantity of fungible tokens derived from the gold bar contract, and addition of a Merkle leaf to update the Merkle root; and (ii) a claim operation performs, in a single atomic ledger transaction, burning of fungible tokens, removal of the Merkle leaf to update the Merkle root, and transfer of the gold bar contract to a claimant.

105. The system of claim 104, wherein the quantity of fungible tokens minted or burned is based on fine gold ounces computed as gross weight multiplied by purity divided by 10,000 with purity stored in basis points.

106. The system of claim 104, wherein each Merkle leaf is computed as a cryptographic hash over canonical-ordered fields including at least an owner identifier, an asset identifier, a custodian identifier, a quantity value, and a nonce.

107. The system of claim 106, wherein the nonce is a cryptographically random value configured to prevent probing of whether a particular owner-asset combination exists in a Merkle tree.

108. The system of claim 104, wherein the gold bar contract stores a Merkle leaf index used to remove the Merkle leaf during claim or forfeiture.

109. The system of claim 104, further comprising supply-invariant logic configured to enforce that a total supply of fungible tokens equals a sum of fine gold ounce values represented by Merkle leaves for collateralized bars.

110. A distributed-ledger system for tokenizing physical assets with public-claim enablement, the system comprising: (a) an asset contract representing a physical asset, the asset contract having at least collateralized and uncollateralized states; (b) a fungible token contract; (c) public-claim enablement logic comprising: (i) a forced-on mode in which, when the asset contract is collateralized, public claim is enabled and non-disableable by an owner, such that any compliance-verified party holding sufficient fungible tokens may claim the asset contract; and (ii) an owner-controlled mode in which, when the asset contract is uncollateralized, an owner selectively enables or disables public claim.

111. The system of claim 110, wherein in the forced-on mode the claim operation burns fungible tokens to remove backing and transfers title to the claimant to enable liquidity and price discovery.

112. A distributed-ledger system for compliance-gated asset operations, comprising: (a) an asset contract representing a physical asset; (b) a compliance proof contract storing a risk level and an expiration time; and (c) access-control logic configured to: (i) during a warning period before expiration, permit actions while generating expiration warnings; (ii) during a grace period after expiration, permit actions while generating warnings; and (iii) after the grace period, enter an expired state in which a first set of actions are blocked and a second set of actions are allowed.

113. The system of claim 112, wherein blocked actions include at least one of deposit requests, consignment requests, claim or title transfer, redemption requests, migration requests, or

collateralization.

114. The system of claim 112, wherein allowed actions include at least one of viewing holdings, transferring fungible tokens, proving reserves, checking balances, or receiving fungible tokens.

115. The system of claim 112, wherein a renewal process updates the compliance proof contract responsive to a compliance provider webhook or oracle message.

116. The system of claim 112, wherein the compliance proof contract stores only non-personally-identifying fields comprising a provider identifier, a reference identifier, a risk level, a country code, and an expiration time.

117. A distributed-ledger system for legal forfeiture of tokenized assets with backing-invariant preservation, comprising: (a) an asset contract representing a physical asset, the asset contract having at least collateralized and uncollateralized states; (b) a fungible token contract; (c) a proof-of-reserves contract maintaining a Merkle root; (d) forfeiture verification logic requiring multi-party authorization based at least on a reference to a legal order document and a document hash; and (e) forfeiture execution logic that, upon satisfaction of a multi-party authorization threshold, executes a forced redemption comprising, if the asset contract is collateralized, removal of a Merkle leaf, burning of fungible tokens in an amount corresponding to the asset contract, and force-transfer of the asset contract to a designated recipient in a single atomic ledger transaction.

118. The system of claim 117, further comprising a hybrid encryption scheme in which a legal order document is encrypted with a symmetric key and the symmetric key is encrypted to a plurality of guardians' public keys, enabling each guardian to independently decrypt and verify the legal order.

119. The system of claim 117, wherein the forced redemption bypasses public-claim enablement settings of the asset contract.

120. The system of claim 117, wherein burned tokens are sourced in priority order from at least one of: an owner balance, an insurance or reserve pool, or a protocol treasury reserve.

121. The system of claim 117, further comprising an immutable execution record stored on-chain for the forced redemption.

122. The system of claim 117, wherein order parameters include at least one of jurisdiction, recipient identity, asset identification, or order type.

123. The system of claim 117, wherein the legal order document is stored off-chain in a content-addressed storage system referenced on-chain.

124. A distributed-ledger system for tokenizing precious metal bars, comprising: (a) a precious metal bar contract storing at least a bar identifier, a vault identifier, a gross weight, and a purity factor; (b) a fungible token contract representing a quantity of fine metal units; (c) a proof-of-reserves contract maintaining a Merkle root representing collateralized bars; and (d)

atomic operation logic configured such that a collateralize operation performs, in a single atomic ledger transaction, a state change to collateralized, minting of a quantity of fungible tokens derived from the precious metal bar contract, and addition of a Merkle leaf to update the Merkle root, and such that a claim operation performs, in a single atomic ledger transaction, burning of fungible tokens, removal of the Merkle leaf to update the Merkle root, and transfer of the precious metal bar contract to a claimant.

125. The system of claim 124, wherein the precious metal comprises at least one of gold, silver, platinum, palladium, or combinations thereof.

126. The system of claim 124, wherein fine metal units are computed as gross weight multiplied by purity divided by 10,000 with purity stored in basis points.

127. A method for maintaining proof-of-reserves for tokenized gold bars, comprising: (a) collateralizing a gold bar contract by executing, in a single transaction, (i) changing a state to collateralized, (ii) minting fungible tokens based on the bar contract, and (iii) adding a Merkle leaf and updating a Merkle root; and (b) claiming the gold bar contract by executing, in a single transaction, (i) burning fungible tokens, (ii) removing the Merkle leaf and updating the Merkle root, and (iii) transferring the gold bar contract to a claimant.

128. The method of claim 127, further comprising computing each Merkle leaf as a hash of canonical-ordered fields including at least an owner identifier, an asset identifier, a custodian identifier, a quantity value, and a nonce.

129. The method of claim 127, further comprising enforcing a supply invariant based on a sum of fine gold ounce values represented by Merkle leaves.

130. The method of claim 127, further comprising verifying claimant compliance proof validity and blocking claim when the compliance proof is expired beyond a grace period.

131. A non-transitory computer-readable medium storing instructions that, when executed by one or more processors, cause performance of the method of claim 127.

132. The system of claim 104, wherein collateralize is an owner-only operation performed without guardian approval.

133. The system of claim 104, wherein the claim operation verifies claimant compliance proof validity prior to permitting claim.

134. The system of claim 104, further comprising a fee routing mechanism configured to route at least one of a protocol fee or a vault fee during claim.

135. The system of claim 104, further comprising a prepaid tiered storage fee escrow contract configured to update a prepaid period and recalculate a tier responsive to a change in ownership during claim.

136. The system of claim 135, wherein a prorated refund is computed for a prior owner and a new prepaid period is created or extended for a claimant.

137. The system of claim 104, further comprising an information-request state in which a vault requests documentation, and further comprising a timeout and resolution state including at least one of denied, canceled, or expired.

138. The system of claim 137, wherein either a vault or a user can cancel a deposit request or redemption request prior to shipment initiation.

139. The system of claim 104, wherein the system is implemented without use of a fiat price oracle for storage fee computation.

140. The system of claim 104, further comprising a redemption workflow in which an asset contract is locked during review and archived upon shipment initiation, and wherein pre-shipment requests are revertible to unlock the asset contract.

141. The system of claim 104, further comprising per-vault Merkle roots and a global Merkle root.

142. A protocol for tokenizing heterogeneous real-world assets on a distributed ledger, comprising: (i) an asset-class registry defining per-class unit semantics and a value-to-unit mapping, (ii) asset instances that reference the registry, and (iii) issuance and lifecycle logic that produces transferable digital units corresponding to the value-to-unit mapping.

143. The protocol of claim 142, wherein multiple asset classes share a common set of protocol components comprising at least one of compliance proofs, proof-of-reserves commitments, fee escrow, and multi-party governance controls.

144. The protocol of claim 142, wherein the value-to-unit mapping is linear and uses a divisor and a quality factor format selected per asset class, to the exclusion of polynomial, exponential, oracle-driven, or scriptable valuation logic.

145. The protocol of claim 142, wherein an asset instance is a conditional claim instrument that confers a right to obtain ownership or control upon satisfaction of recorded conditions including compliance eligibility and custodian attestation.

146. The protocol of claim 142, wherein the proof-of-reserves commitment comprises per-custodian roots and an aggregate root.

147. The protocol of claim 142, wherein fractionalization is enabled by an optional sub-claim construct in which sub-claim base quantities sum to the parent base quantity.

148. A distributed-ledger protocol for tokenizing multiple classes of real-world assets, comprising: (a) an asset class registry storing, for each asset class, (i) base quantity units, (ii) a quality factor format, (iii) a divisor, and (iv) a linear unit-value formula; (b) an asset contract storing an asset class identifier, a base quantity, and a quality factor for an asset instance; (c) unit-value computation logic configured to compute a unit value for the asset contract by applying the linear unit-value formula using the divisor and the quality factor format; and (d) minting logic configured to mint fungible tokens based on the unit value.

149. The protocol of claim 148, wherein the asset class registry supports a precious metals class in which base quantity units are troy ounces, the quality factor format is basis points, and the divisor is 10,000.

150. The protocol of claim 148, wherein the asset class registry supports a real estate class in which base quantity units are square feet and the quality factor format is a decimal factor.

151. The protocol of claim 148, wherein the linear unit-value formula is limited to multiplicative combinations of the base quantity and the quality factor divided by the divisor, to the exclusion of polynomial, exponential, oracle-driven, or scriptable valuation logic.

152. The protocol of claim 148, further comprising collateralization logic configured to cause an asset contract to become collateralized and mint fungible tokens equal to the unit value.

153. The protocol of claim 152, further comprising proof-of-reserves logic configured to maintain a Merkle root over collateralized asset contracts.

154. The protocol of claim 153, wherein each Merkle leaf includes a cryptographically random salt to reduce probing of asset identifiers or ownership.

155. The protocol of claim 148, wherein an asset class in the asset class registry corresponds to precious metal bars and is implemented by an asset contract storing a bar identifier, a vault identifier, a gross weight, and a purity factor stored in basis points, and wherein unit value corresponds to fine metal units computed using the divisor.

156. A distributed-ledger protocol for tokenizing multiple classes of real-world assets using shared protocol components, comprising: (a) an asset class registry storing, for each asset class, unit semantics and a unit-value formula; (b) asset contracts referencing the asset class registry; (c) a proof-of-reserves contract maintaining a cryptographic commitment over collateralized asset contracts; (d) a reusable compliance proof contract storing a risk level and an expiration time; (e) a prepaid tiered storage fee escrow contract computing percentage-based fees as a function of base quantity; and (f) a guardian approval system configured to govern at least one state-changing action, the guardian approval system providing action-specific guardian sets and action-specific thresholds.

157. The protocol of claim 156, wherein the guardian approval system provides a sequential approval queue with timeout expiration and cancel requests to unblock the queue.

158. The protocol of claim 156, wherein the compliance proof contract blocks selected title-changing actions when expired beyond a grace period while allowing selected non-title-changing actions.

159. The protocol of claim 156, wherein the prepaid tiered storage fee escrow contract maintains one prepaid period per user per asset type per custodian.

160. The protocol of claim 156, further comprising deposit and consignment workflows that separately represent physical movement to a custodian and on-chain placement into custody.

161. The protocol of claim 156, further comprising an information-request state machine with an on-chain information_request state and an on-chain resolution state including at least one of denied, canceled, or expired.

162. The protocol of claim 156, further comprising legal forfeiture execution logic configured to force-transfer an asset contract upon multi-party verification and preserve a backing invariant by burning fungible tokens equal to the unit value when the asset contract is collateralized.

163. The protocol of claim 162, wherein a forfeiture document is stored off-chain and referenced on-chain by a document hash and content identifier.

164. The protocol of claim 156, wherein the guardian approval system governs at least one of asset-class registry updates, proof-of-reserves root updates, forfeiture operations, emergency pause operations, or migration operations.

165. A distributed-ledger system for representing real-world assets as conditional claim instruments, comprising: (a) an asset contract representing a right to obtain ownership or control of a corresponding real-world asset upon satisfaction of one or more conditions; (b) compliance verification logic configured to verify at least one condition for claiming, wherein the at least one condition includes a valid compliance proof; and (c) ownership transfer logic configured to transfer the asset contract to a claimant upon satisfaction of the one or more conditions without requiring intermediary involvement until physical redemption.

166. The system of claim 165, wherein the one or more conditions further include custodian attestation or completion of a custody transfer workflow.

167. The system of claim 165, wherein the one or more conditions further include payment of applicable redemption or transport fees.

168. The system of claim 165, wherein satisfaction of the one or more conditions is evidenced by on-chain state transitions and/or custodian attestation.

169. The system of claim 165, wherein the asset contract avoids custodial IOU structures such that the asset contract represents a direct claim against a specific physical asset rather than a claim against an issuer.

170. A method for tokenizing multiple classes of real-world assets on a distributed ledger, comprising: (a) storing, in an asset class registry, a linear unit-value formula and a divisor for an asset class; (b) creating an asset contract storing an asset class identifier, a base quantity, and a quality factor; (c) computing a unit value by applying the linear unit-value formula using the divisor and the quality factor; and (d) minting fungible tokens based on the unit value.

171. The method of claim 170, further comprising collateralizing the asset contract and recording the asset contract in a proof-of-reserves Merkle structure.

172. The method of claim 170, further comprising gating at least one of claim, transfer, consignment, or redemption on a compliance proof validity state.

173. The method of claim 170, further comprising applying a guardian approval queue with timeout and cancel for governance actions.

174. A non-transitory computer-readable medium storing instructions that, when executed, cause performance of the method of claim 170.

175. The protocol of claim 148, wherein the protocol maintains a backing invariant such that total fungible token supply equals a sum of unit values for collateralized asset contracts.

176. The protocol of claim 148, wherein the asset class registry stores a supportsFractionalization indicator and, when enabled, permits creation of multiple sub-claim contracts associated with a parent asset contract.

177. The protocol of claim 176, wherein a sum of base quantities of the multiple sub-claim contracts equals a base quantity of the parent asset contract.

178. The protocol of claim 148, wherein the protocol restricts unit-value computation to linear formulas as defined in the asset class registry.

179. The protocol of claim 156, wherein the proof-of-reserves contract maintains per-custodian Merkle roots and a global aggregate root.

180. The protocol of claim 148, wherein the asset class registry permits dynamic registration of new asset classes through a guardian-approved governance action without requiring code changes to the protocol.

181. A method for resolving blocked sequential-execution queues in a guardian-governed smart contract system, comprising: (a) detecting that a pending governance request has remained unresolved past a configurable timeout period; (b) applying one of two resolution paths: (i) an expiration path, wherein the pending request is marked expired and cleaned from the queue, advancing the next-executable-request pointer; or (ii) a cancellation path, wherein the pending request is cancelled out-of-order upon meeting a disapproval threshold, without requiring advancement of the sequential pointer; (c) resuming sequential execution of subsequent queued requests after resolution, wherein the configurable timeout period defaults to seven days and is adjustable per contract deployment.

182. The governance control plane of claim 61, wherein the system enforces a threshold cap rule requiring that any approval threshold must be at most one less than the total guardian count, and wherein the system automatically decrements any threshold that violates the cap rule upon guardian removal.

183. The system of claim 98, wherein each Merkle leaf is computed as a SHA-256 hash over a canonical field order comprising: owner identifier, bar identifier, vault identifier, fine gold weight in troy ounces, and a 256-bit random salt nonce, concatenated in that fixed order.

184. The system of claim 98, further comprising a user-facing verification interface that exposes a verifyAllocation function accepting a user identifier, an amount, and a Merkle proof array, and returning a boolean confirmation that the user's allocation is included in the current

proof-of-reserves Merkle root.

185. The system of claim 107, wherein the redemption workflow further comprises a revert mechanism operable before physical shipment, wherein either the vault custodian or the redeeming user may trigger reversion, causing the locked non-fungible token to be unlocked and the fungible token burn to be reversed, restoring the pre-redemption state.

186. The protocol of claim 148, wherein claiming a conditional-claim instrument requires satisfaction of anti-money-laundering screening and sanctions-list screening as preconditions, and wherein ongoing ownership verification is enforced by periodic re-screening at intervals determined by a risk-based KYC expiration schedule comprising a plurality of risk tiers with different expiration periods and a grace period before access restriction.